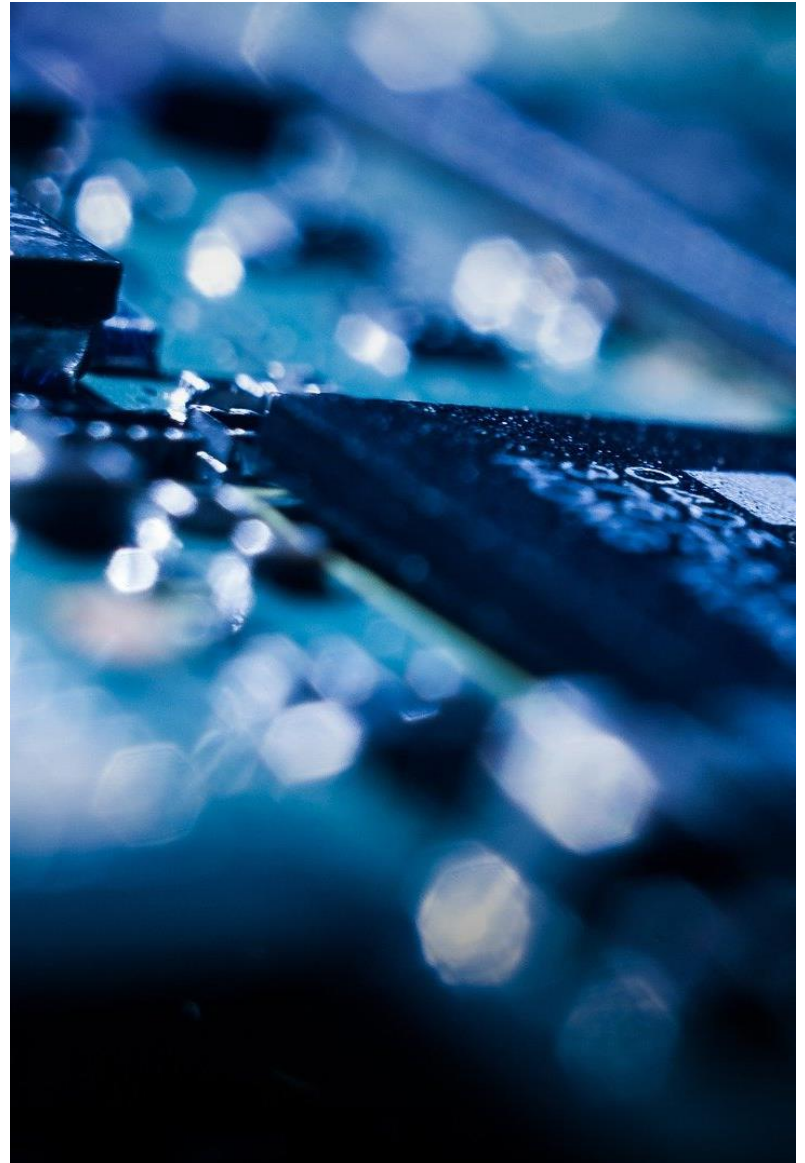


Programmierung eingebetteter Systeme

Peripherie

Prof. Dr. Henning Trsek

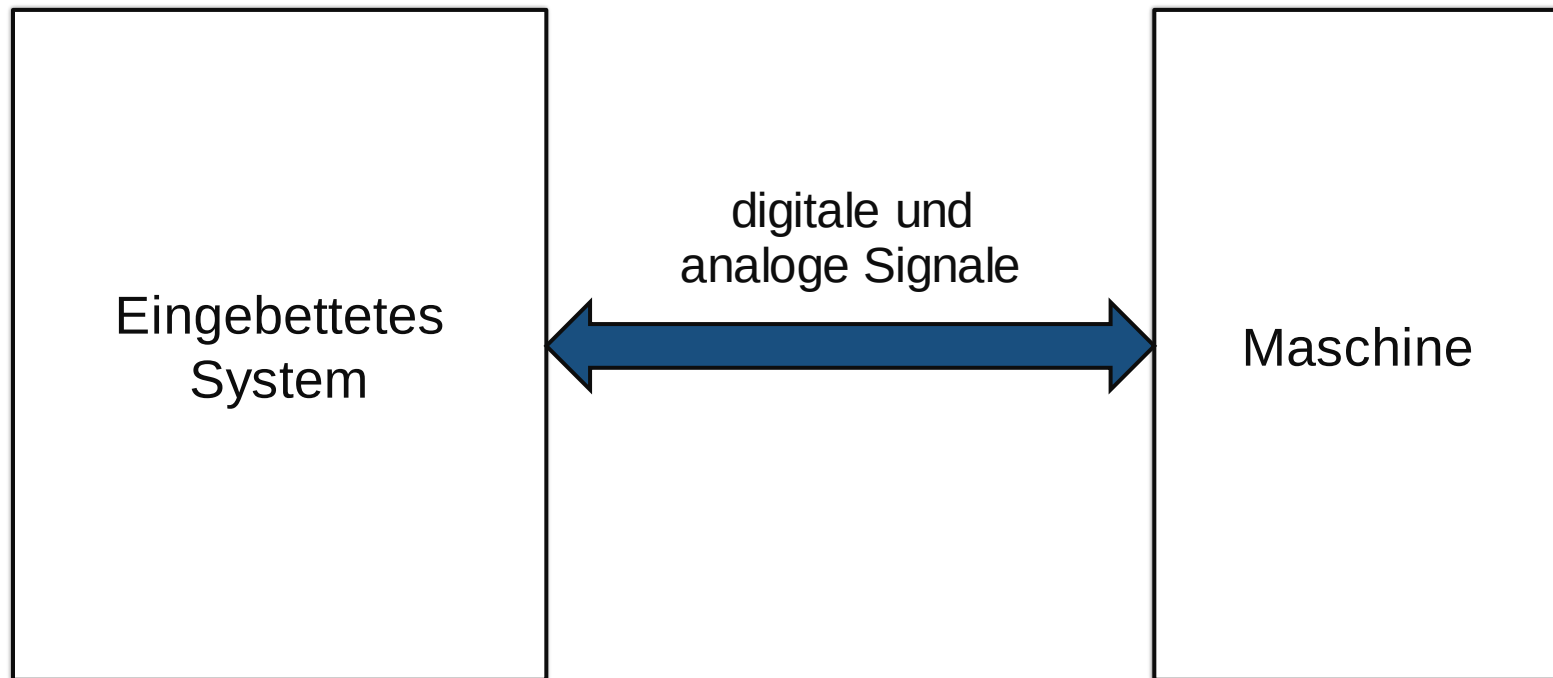
Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick – Peripherie

- ▶ Anbindung Peripherie
- ▶ Digitale Ausgabe
- ▶ Digitale Eingabe
- ▶ Beispiel Tabellenverarbeitung
- ▶ Logische Befehle

Schnittstellen zur Außenwelt



Schnittstellen zur Außenwelt

Figure 1-1 MC9S12DP512 Block Diagram

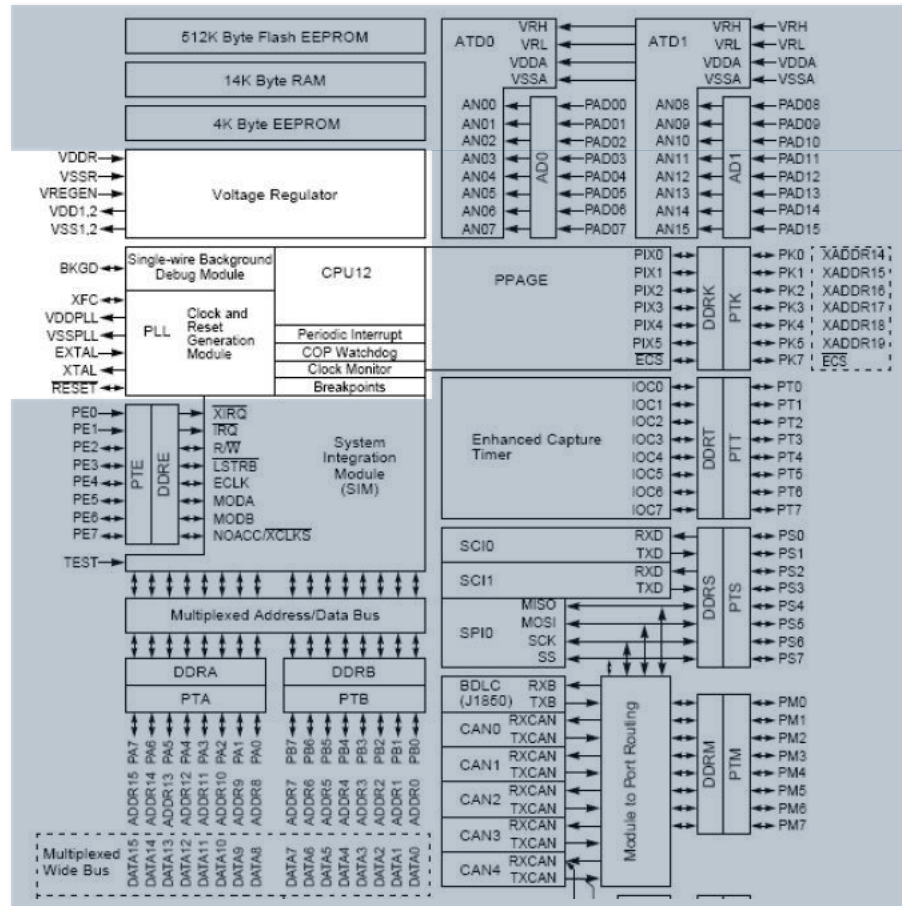
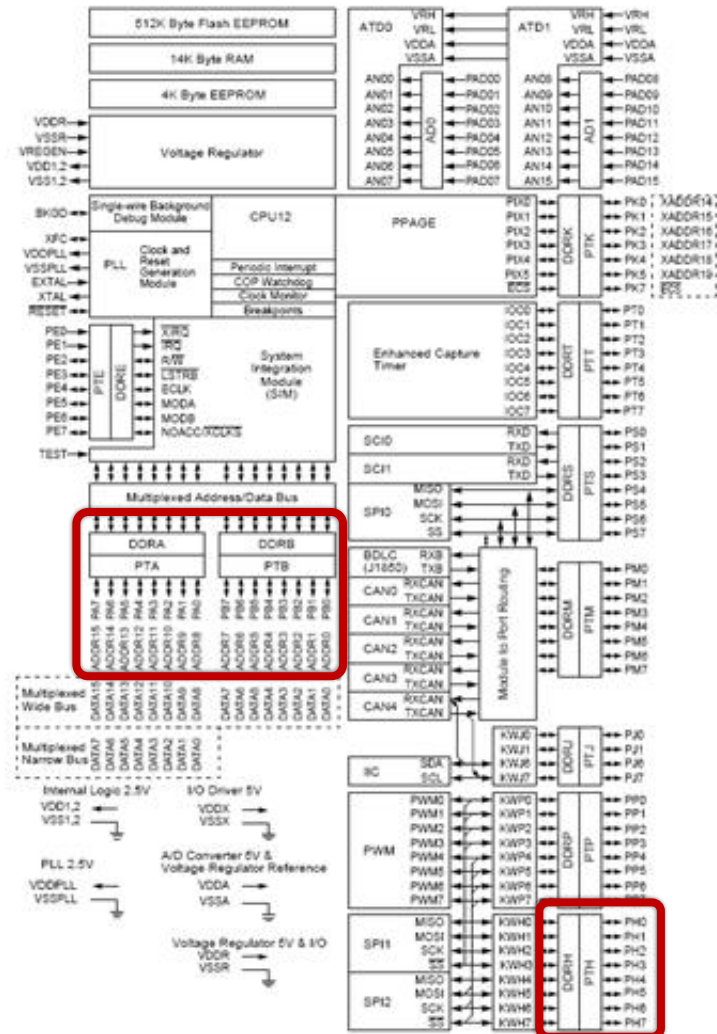
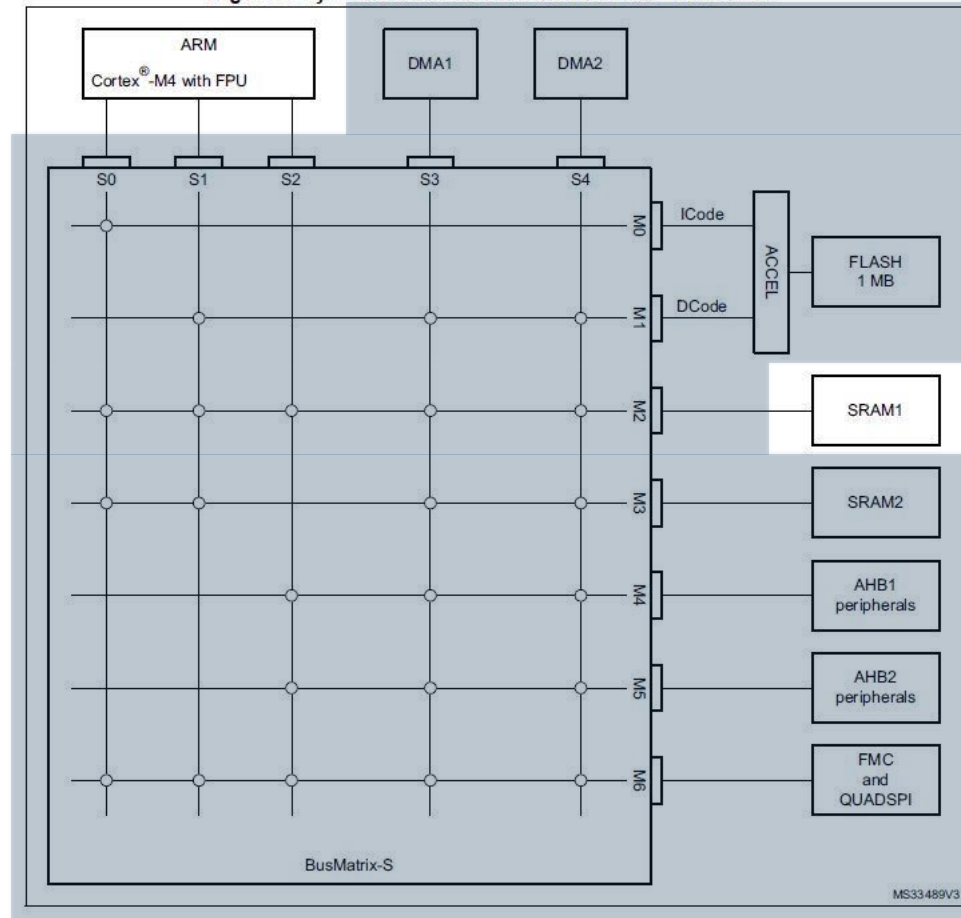


Figure 1-1 MC9S12DP512 Block Diagram



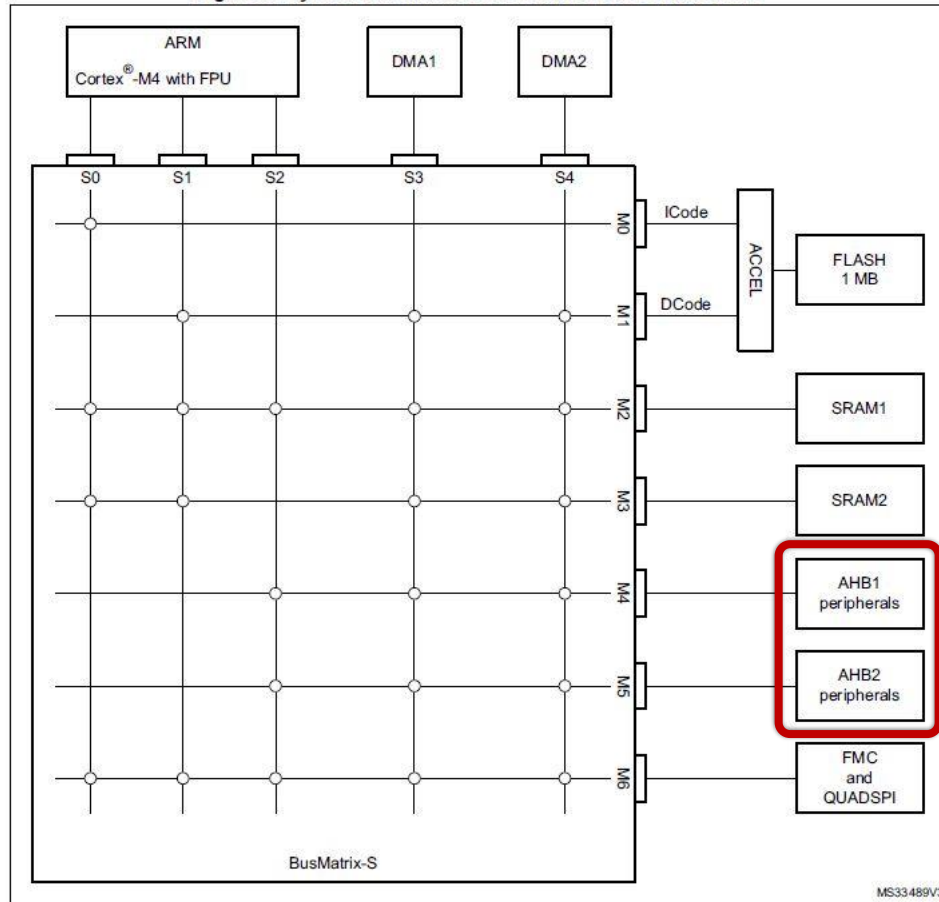
Schnittstellen zur Außenwelt

Figure 1. System architecture for STM32L47x/L48x devices

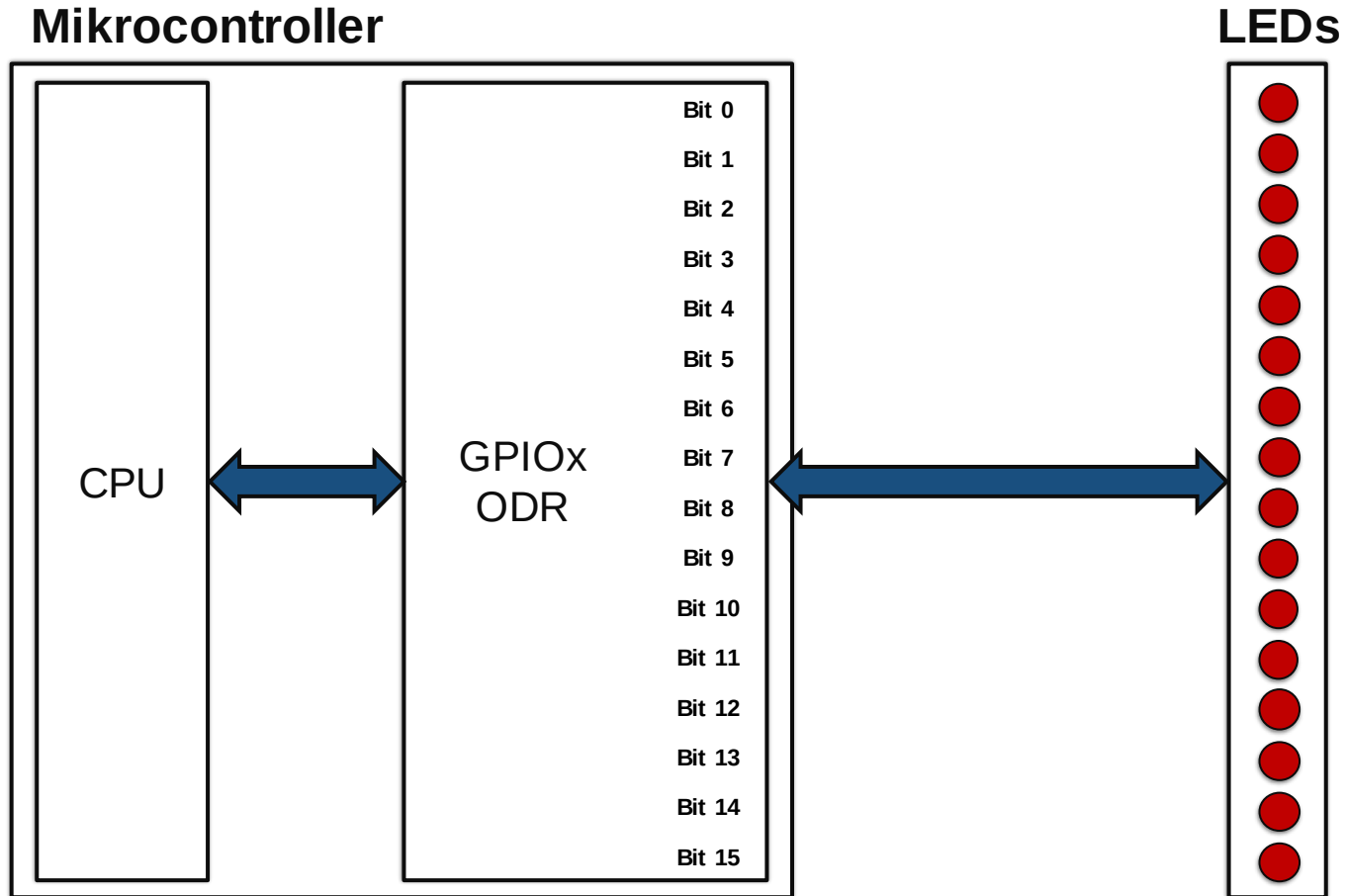


Schnittstellen zur Außenwelt

Figure 1. System architecture for STM32L47x/L48x devices

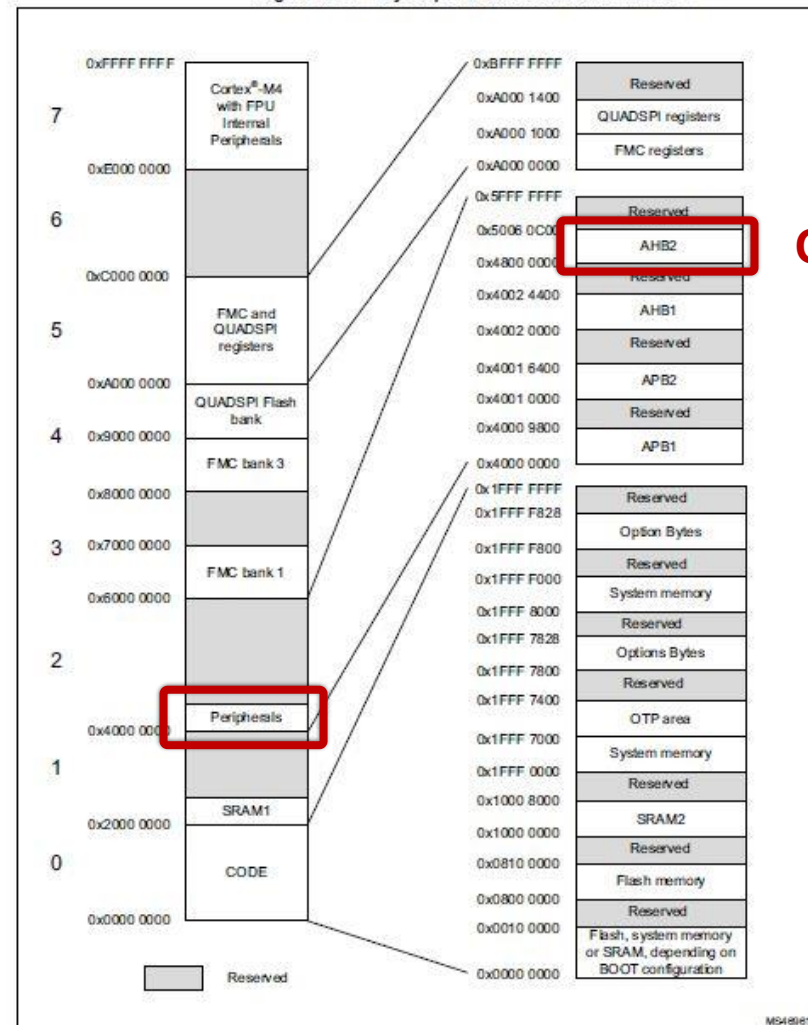


Schnittstellen zur Außenwelt



Schnittstellen zur Außenwelt

Figure 3. Memory map for STM32L47x/L48x devices



GPIO Register

Schnittstellen zur Außenwelt

Table 1. STM32L47x/L48x devices memory map and peripheral register boundary addresses⁽¹⁾

Bus	Boundary address	Size (bytes)	Peripheral	Peripheral register map
AHB2	0x5006 0800 - 0x5006 0BFF	1 KB	RNG	Section 27.7.4: RNG register map
	0x5006 0400 - 0x5006 07FF	1 KB	Reserved	-
	0x5006 0000 - 0x5006 03FF	1 KB	AES ⁽²⁾	Section 28.7.18: AES register map
	0x5004 0400 - 0x5005 FFFF	127 KB	Reserved	-
	0x5004 0000 - 0x5004 03FF	1 KB	ADC	Section 18.8: ADC register map
	0x5000 0000 - 0x5003 FFFF	256 KB	OTG_FS ⁽³⁾	Section 47.15.57: OTG_FS register map
	0x4800 2000 - 0x4FFF FFFF	~127 MB	Reserved	-
	0x4800 1C00 - 0x4800 1FFF	1 KB	GPIOH	Section 8.5.13: GPIO register map
	0x4800 1800 - 0x4800 1BFF	1 KB	GPIOG	Section 8.5.13: GPIO register map
	0x4800 1400 - 0x4800 17FF	1 KB	GPIOF	Section 8.5.13: GPIO register map
	0x4800 1000 - 0x4800 13FF	1 KB	GPIOE	Section 8.5.13: GPIO register map
	0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD	Section 8.5.13: GPIO register map
	0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC	Section 8.5.13: GPIO register map
	0x4800 0400 - 0x4800 07FF	1 KB	GPIOB	Section 8.5.13: GPIO register map
	0x4800 0000 - 0x4800 03FF	1 KB	GPIOA	Section 8.5.13: GPIO register map
	0x4002 4400 - 0x47FF FFFF	~127 MB	Reserved	-

GPIOA

Schnittstellen zur Außenwelt

Table 1. STM32L47x/L48x devices memory map and peripheral register boundary addresses⁽¹⁾

Bus	Boundary address	Size (bytes)	Peripheral	Peripheral register map
AHB2	0x5006 0800 - 0x5006 0BFF	1 KB	RNG	Section 27.7.4: RNG register map
	0x5006 0400 - 0x5006 07FF	1 KB	Reserved	-
	0x5006 0000 - 0x5006 03FF	1 KB	AES ⁽²⁾	Section 28.7.18: AES register map
	0x5004 0400 - 0x5005 FFFF	127 KB	Reserved	-
	0x5004 0000 - 0x5004 03FF	1 KB	ADC	Section 18.8: ADC register map
	0x5000 0000 - 0x5003 FFFF	256 KB	OTG_FS ⁽³⁾	Section 47.15.57: OTG_FS register map
	0x4800 2000 - 0x4FFF FFFF	~127 MB	Reserved	-
	0x4800 1C00 - 0x4800 1FFF	1 KB	GPIOH	Section 8.5.13: GPIO register map
	0x4800 1800 - 0x4800 1BFF	1 KB	GPIOG	Section 8.5.13: GPIO register map
	0x4800 1400 - 0x4800 17FF	1 KB	GPIOF	Section 8.5.13: GPIO register map
	0x4800 1000 - 0x4800 13FF	1 KB	GPIOE	Section 8.5.13: GPIO register map
	0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD	Section 8.5.13: GPIO register map
	0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC	Section 8.5.13: GPIO register map
	0x4800 0400 - 0x4800 07FF	1 KB	GPIOB	Section 8.5.13: GPIO register map
	0x4800 0000 - 0x4800 03FF	1 KB	GPIOA	Section 8.5.13: GPIO register map
	0x4002 4400 - 0x47FF FFFF	~127 MB	Reserved	-

GPIOC

Schnittstellen zur Außenwelt

8.5.13 GPIO register map

The following table gives the GPIO register map and reset values.

Table 40. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODE[15:0]	MODE[15:0]	MODE[14:0]	MODE[14:0]	MODE[13:0]	MODE[13:0]	MODE[12:0]	MODE[12:0]	MODE[11:0]	MODE[11:0]	MODE[10:0]	MODE[10:0]	MODE[9:0]	MODE[9:0]	MODE[8:0]	MODE[8:0]	MODE[7:0]	MODE[7:0]	MODE[6:0]	MODE[6:0]	MODE[5:0]	MODE[5:0]	MODE[4:0]	MODE[4:0]	MODE[3:0]	MODE[3:0]	MODE[2:0]	MODE[2:0]	MODE[1:0]	MODE[1:0]	MODE[0:0]	
	Reset value	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x00	GPIOB_MODER	MODE[15:0]	MODE[15:0]	MODE[14:0]	MODE[14:0]	MODE[13:0]	MODE[13:0]	MODE[12:0]	MODE[12:0]	MODE[11:0]	MODE[11:0]	MODE[10:0]	MODE[10:0]	MODE[9:0]	MODE[9:0]	MODE[8:0]	MODE[8:0]	MODE[7:0]	MODE[7:0]	MODE[6:0]	MODE[6:0]	MODE[5:0]	MODE[5:0]	MODE[4:0]	MODE[4:0]	MODE[3:0]	MODE[3:0]	MODE[2:0]	MODE[2:0]	MODE[1:0]	MODE[1:0]	MODE[0:0]	
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x00	GPIOx_MODER (where x = C..J)	MODE[15:0]	MODE[15:0]	MODE[14:0]	MODE[14:0]	MODE[13:0]	MODE[13:0]	MODE[12:0]	MODE[12:0]	MODE[11:0]	MODE[11:0]	MODE[10:0]	MODE[10:0]	MODE[9:0]	MODE[9:0]	MODE[8:0]	MODE[8:0]	MODE[7:0]	MODE[7:0]	MODE[6:0]	MODE[6:0]	MODE[5:0]	MODE[5:0]	MODE[4:0]	MODE[4:0]	MODE[3:0]	MODE[3:0]	MODE[2:0]	MODE[2:0]	MODE[1:0]	MODE[1:0]	MODE[0:0]	
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x04	GPIOx_OTYPER (where x = A..J)	OT[15]	OT[15]	OT[14]	OT[14]	OT[13]	OT[13]	OT[12]	OT[12]	OT[11]	OT[11]	OT[10]	OT[10]	OT[9]	OT[9]	OT[8]	OT[8]	OT[7]	OT[7]	OT[6]	OT[6]	OT[5]	OT[5]	OT[4]	OT[4]	OT[3]	OT[3]	OT[2]	OT[2]	OT[1]	OT[1]	OT[0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOA_OSPEEDR	OSPEED[15:0]	OSPEED[15:0]	OSPEED[14:0]	OSPEED[14:0]	OSPEED[13:0]	OSPEED[13:0]	OSPEED[12:0]	OSPEED[12:0]	OSPEED[11:0]	OSPEED[11:0]	OSPEED[10:0]	OSPEED[10:0]	OSPEED[9:0]	OSPEED[9:0]	OSPEED[8:0]	OSPEED[8:0]	OSPEED[7:0]	OSPEED[7:0]	OSPEED[6:0]	OSPEED[6:0]	OSPEED[5:0]	OSPEED[5:0]	OSPEED[4:0]	OSPEED[4:0]	OSPEED[3:0]	OSPEED[3:0]	OSPEED[2:0]	OSPEED[2:0]	OSPEED[1:0]	OSPEED[1:0]	OSPEED[0:0]	
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOx_OSPEEDR (where x = B..J)	OSPEED[15:0]	OSPEED[15:0]	OSPEED[14:0]	OSPEED[14:0]	OSPEED[13:0]	OSPEED[13:0]	OSPEED[12:0]	OSPEED[12:0]	OSPEED[11:0]	OSPEED[11:0]	OSPEED[10:0]	OSPEED[10:0]	OSPEED[9:0]	OSPEED[9:0]	OSPEED[8:0]	OSPEED[8:0]	OSPEED[7:0]	OSPEED[7:0]	OSPEED[6:0]	OSPEED[6:0]	OSPEED[5:0]	OSPEED[5:0]	OSPEED[4:0]	OSPEED[4:0]	OSPEED[3:0]	OSPEED[3:0]	OSPEED[2:0]	OSPEED[2:0]	OSPEED[1:0]	OSPEED[1:0]	OSPEED[0:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	GPIOA_PUPDR	PUPD[15:0]	PUPD[15:0]	PUPD[14:0]	PUPD[14:0]	PUPD[13:0]	PUPD[13:0]	PUPD[12:0]	PUPD[12:0]	PUPD[11:0]	PUPD[11:0]	PUPD[10:0]	PUPD[10:0]	PUPD[9:0]	PUPD[9:0]	PUPD[8:0]	PUPD[8:0]	PUPD[7:0]	PUPD[7:0]	PUPD[6:0]	PUPD[6:0]	PUPD[5:0]	PUPD[5:0]	PUPD[4:0]	PUPD[4:0]	PUPD[3:0]	PUPD[3:0]	PUPD[2:0]	PUPD[2:0]	PUPD[1:0]	PUPD[1:0]	PUPD[0:0]	
	Reset value	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	GPIOB_PUPDR	PUPD[15:0]	PUPD[15:0]	PUPD[14:0]	PUPD[14:0]	PUPD[13:0]	PUPD[13:0]	PUPD[12:0]	PUPD[12:0]	PUPD[11:0]	PUPD[11:0]	PUPD[10:0]	PUPD[10:0]	PUPD[9:0]	PUPD[9:0]	PUPD[8:0]	PUPD[8:0]	PUPD[7:0]	PUPD[7:0]	PUPD[6:0]	PUPD[6:0]	PUPD[5:0]	PUPD[5:0]	PUPD[4:0]	PUPD[4:0]	PUPD[3:0]	PUPD[3:0]	PUPD[2:0]	PUPD[2:0]	PUPD[1:0]	PUPD[1:0]	PUPD[0:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	GPIOx_PUPDR (where x = C..J)	PUPD[15:0]	PUPD[15:0]	PUPD[14:0]	PUPD[14:0]	PUPD[13:0]	PUPD[13:0]	PUPD[12:0]	PUPD[12:0]	PUPD[11:0]	PUPD[11:0]	PUPD[10:0]	PUPD[10:0]	PUPD[9:0]	PUPD[9:0]	PUPD[8:0]	PUPD[8:0]	PUPD[7:0]	PUPD[7:0]	PUPD[6:0]	PUPD[6:0]	PUPD[5:0]	PUPD[5:0]	PUPD[4:0]	PUPD[4:0]	PUPD[3:0]	PUPD[3:0]	PUPD[2:0]	PUPD[2:0]	PUPD[1:0]	PUPD[1:0]	PUPD[0:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	GPIOx_IDR (where x = A..J)	ID[15]	ID[15]	ID[14]	ID[14]	ID[13]	ID[13]	ID[12]	ID[12]	ID[11]	ID[11]	ID[10]	ID[10]	ID[9]	ID[9]	ID[8]	ID[8]	ID[7]	ID[7]	ID[6]	ID[6]	ID[5]	ID[5]	ID[4]	ID[4]	ID[3]	ID[3]	ID[2]	ID[2]	ID[1]	ID[1]	ID[0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Schnittstellen zur Außenwelt

Table 40. GPIO register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x14	GPIOx_ODR (where x = A..I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	GPIOx_BSRR (where x = A..I)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1C	GPIOx_LCKR (where x = A..I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LCKK	LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	GPIOx_AFR1 (where x = A..I)	AFSEL7[3:0]				AFSEL6[3:0]				AFSEL5[3:0]				AFSEL4[3:0]				AFSEL3[3:0]				AFSEL2[3:0]				AFSEL1[3:0]				AFSEL0[3:0]			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x24	GPIOx_AFRH (where x = A..I)	AFSEL15[3:0]				AFSEL14[3:0]				AFSEL13[3:0]				AFSEL12[3:0]				AFSEL11[3:0]				AFSEL10[3:0]				AFSEL9[3:0]				AFSEL8[3:0]			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x28	GPIOx_BRR (where x = A..I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x2C	GPIOx_ASCR (where x = A..H)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ASC15	ASC14	ASC13	ASC12	ASC11	ASC10	ASC9	ASC8	ASC7	ASC6	ASC5	ASC4	ASC3	ASC2	ASC1	ASC0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Output
Register

Schnittstellen zur Außenwelt

Beispiel: GPIOC, Output-Register ansprechen?

GPIOC liegt auf den Adressen 0x4800 0800 – 0x4800 0BFF

0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC	Section 8.5.13: GPIO register map
0x4800 0400 - 0x4800 07FF	1 KB	GPIOB	Section 8.5.13: GPIO register map

GPIOC

GPIO-Output-Register haben den Offset 0x14

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x14	GPIOx_ODR (where x = A..I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	GPIOx_BSRR (where x = A..I)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Output Data Register

Schnittstellen zur Außenwelt

Beispiel: GPIOC, Output-Register ansprechen?

GPIOC liegt auf den Adressen 0x4800 0800 – 0x4800 0BFF

0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC	Section 8.5.13: GPIO register map
0x4800 0400 - 0x4800 07FF	1 KB	GPIOB	Section 8.5.13: GPIO register map

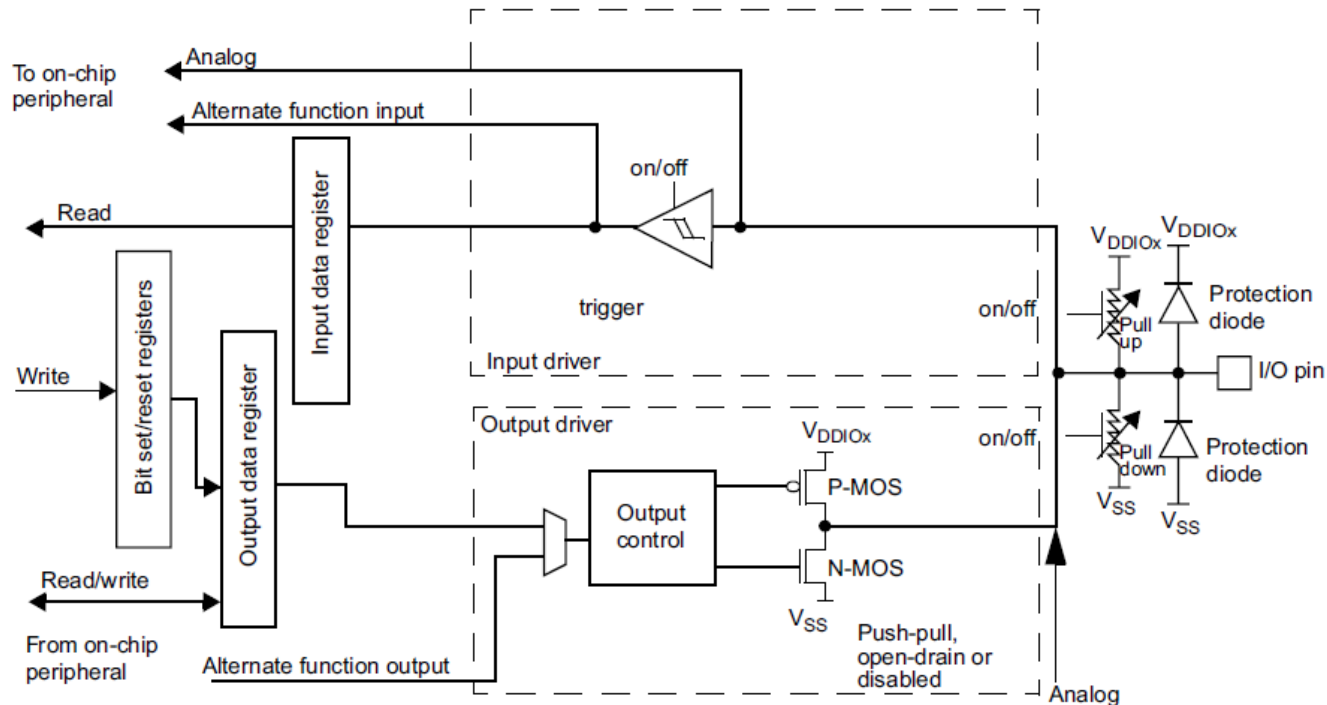
GPIO-Output-Register haben den Offset 0x14

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x14	GPIOx_ODR (where x = A..I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	GPIOx_BSRR (where x = A..I)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

→ **0x4800 0800 + 0x14 = 0x4800 0814**

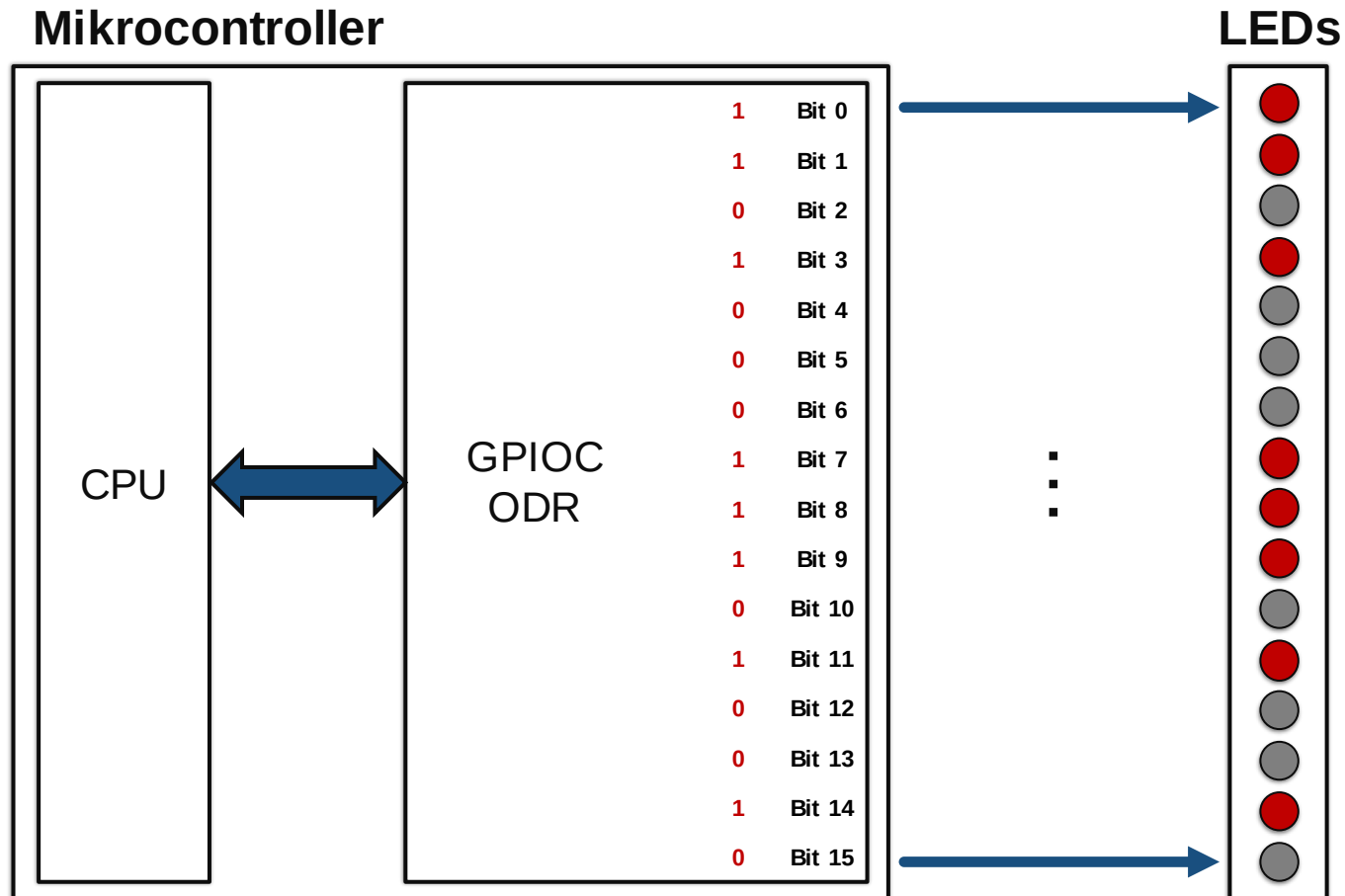
Schnittstellen zur Außenwelt

GPIO-Pin Struktur



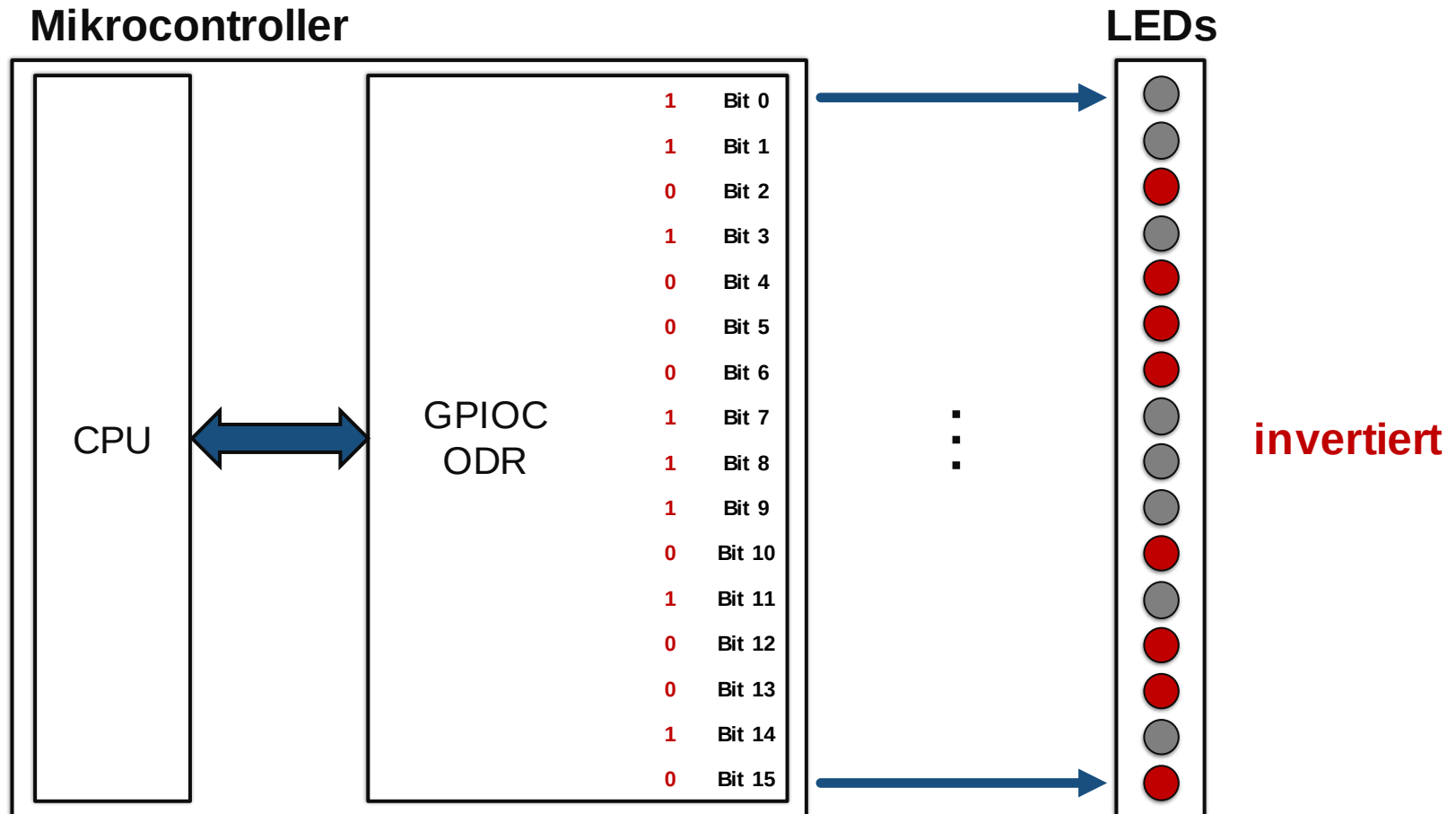
Schnittstellen zur Außenwelt

Beispiel: GPIOC, Pin 0 bis Pin 15 als Output konfiguriert



Schnittstellen zur Außenwelt

Beispiel: GPIOC, Pin 0 bis Pin 15 als Output konfiguriert



Schnittstellen zur Außenwelt

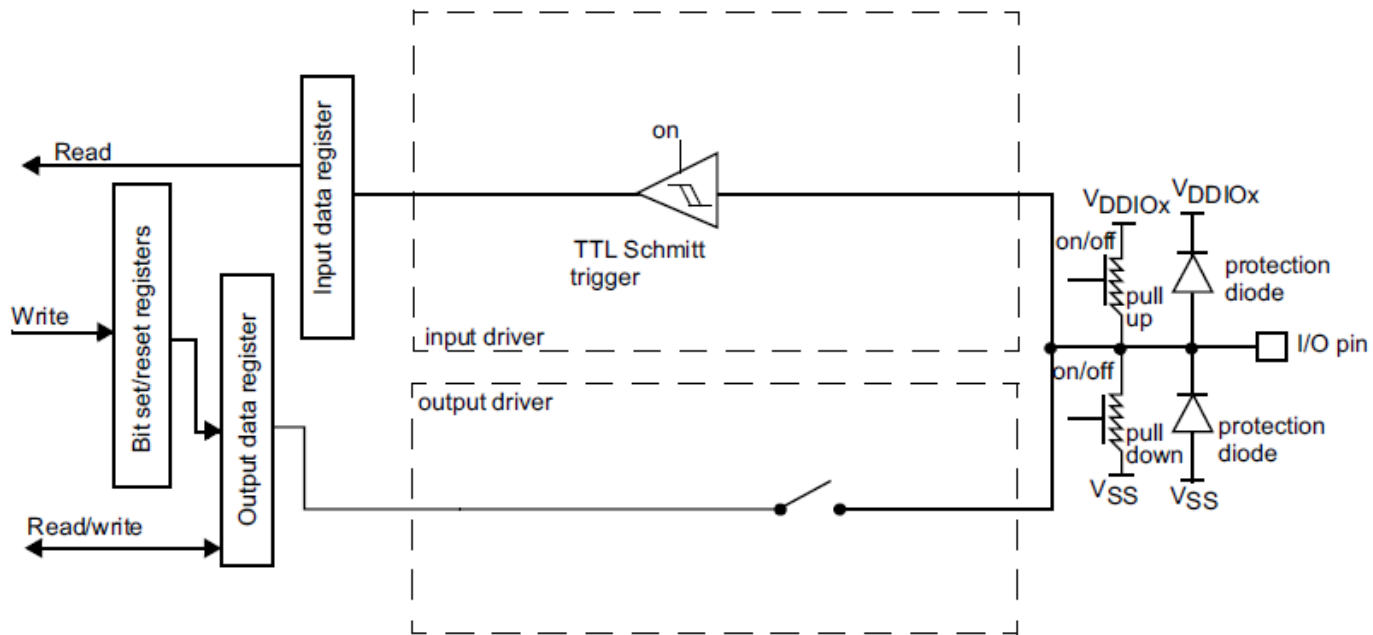
Schnittstellen zur Außenwelt

Input-Register

0x0C	GPIOB_PUPDR	PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]		PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	
0x0C	GPIOx_PUPDR (where x = C..I)	PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]		PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	GPIOx_IDR (where x = A..I)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
	Reset value																	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	

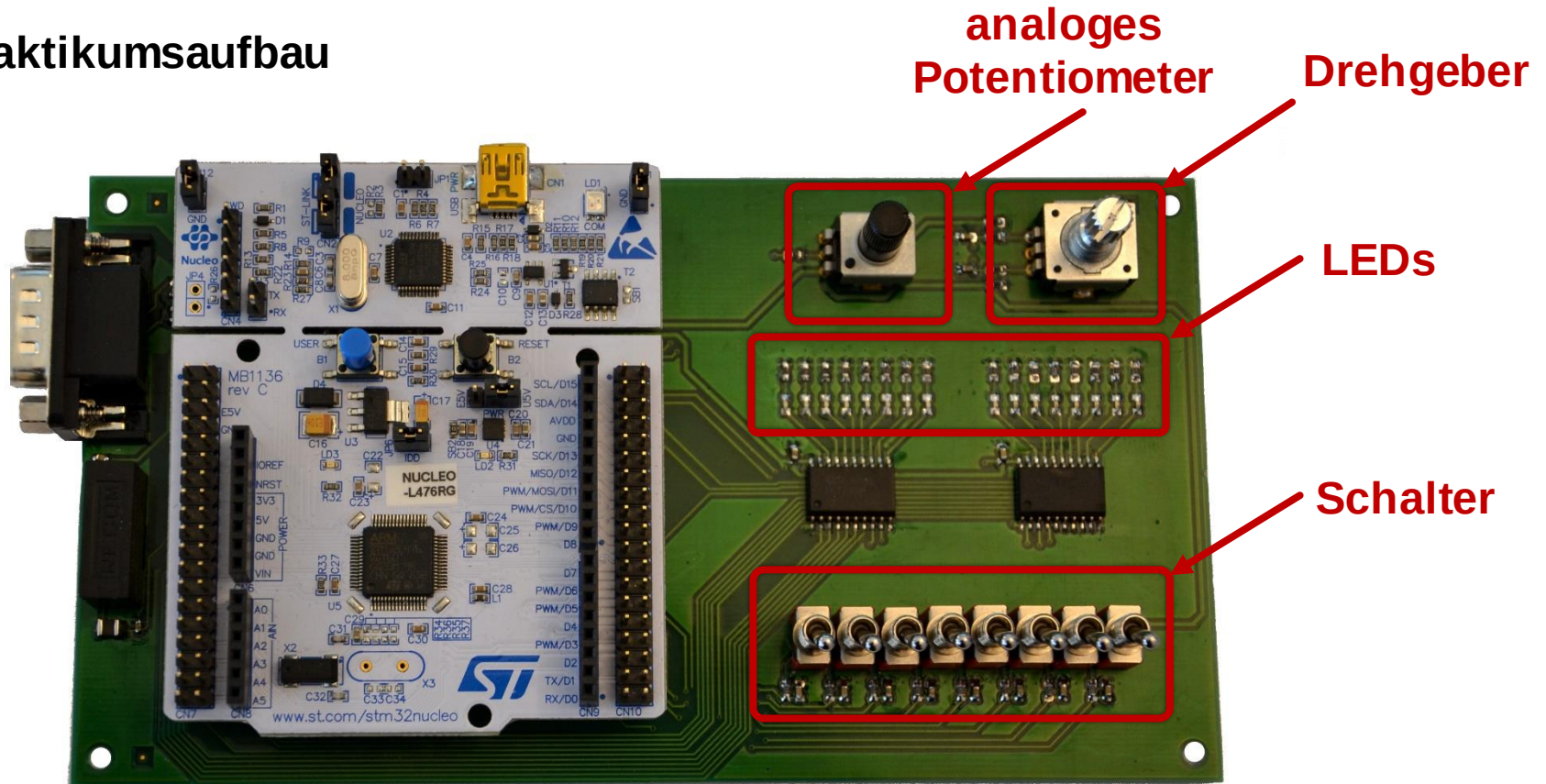
Schnittstellen zur Außenwelt

Input Floating / Pull-Up / Pull-Down Configurations



Schnittstellen zur Außenwelt

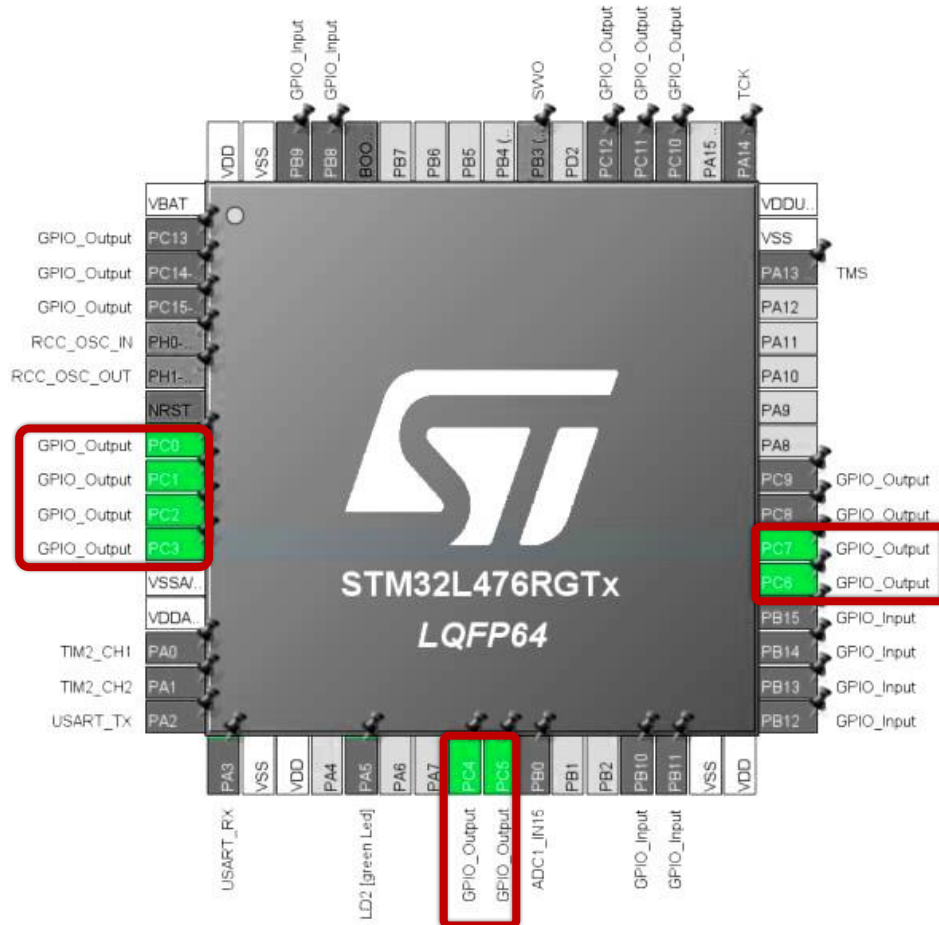
Praktikumsaufbau



Digitale Ausgabe (Digital Out)

Digitale Ausgabe (Digital Out)

Beispiel: GPIOC, Pin 0 bis Pin 7 als Output verwenden



Digitale Ausgabe (Digital Out)

Wie können wir die Output Register erreichen?

Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr    r0,    =#GPIOC_ODR  
      ldr    r1,    [r0]  
      mov    r2,    #0xFF  
      orr    r1,    r2  
      str    r1,    [r0]
```

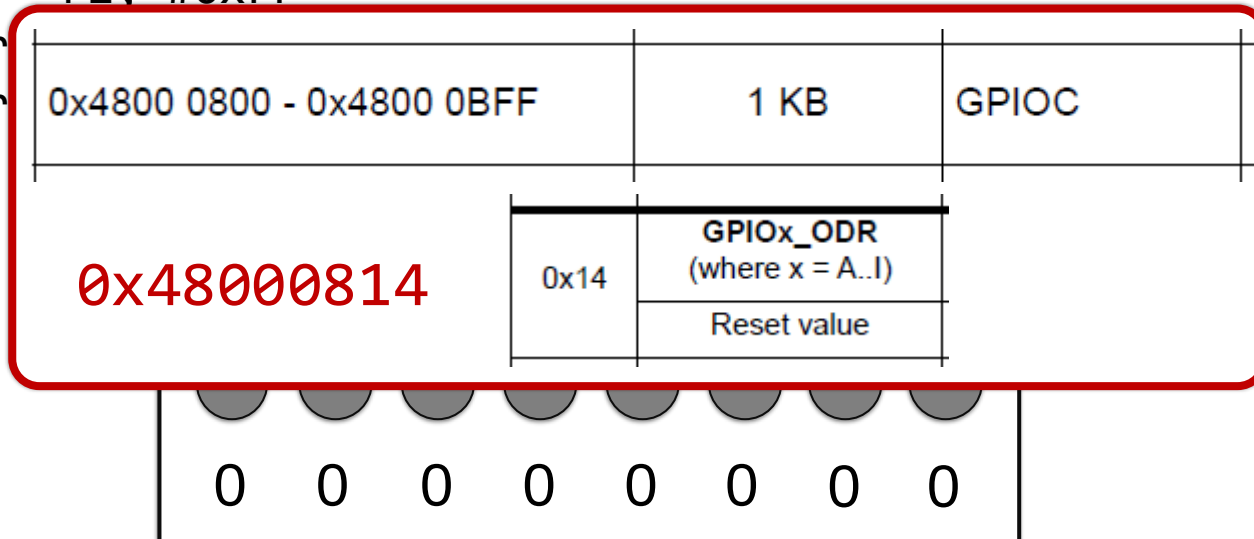


Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

`.equ GPIOC_ODR, 0x48000814` ← Adresse: GPIOC, Output-Register

```
prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, [r0]
      mov  r2, #0xFF
      orr  r1, r2
      str  r1, [r0]
```



Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR ← 0x48000814 in Register R0  
      ldr  r1, [r0]  
      mov  r2, #0xFF  
      orr  r1, r2  
      str  r1, [r0]
```



Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr    r0,    =#GPIOC_ODR
      ldr    r1,    [r0]
      mov    r2,    #0xFF
      orr    r1,    r2
      str    r1,    [r0]
```

urspr. Inhalt des Output-Registers in Register R1



Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, [r0]
      mov  r2, #0xFF
      orr  r1, r2
      str  r1, [r0]
```

0xFF (binär 11111111)
in Register R2



Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, [r0]
      mov  r2, #0xFF
      orr  r1, r2
      str  r1, [r0]
```

0xFF mit Inhalt von R1 verodern,
damit Bits 8–15 unverändert bleiben



Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, [r0]
      mov  r2, #0xFF
      orr  r1, r2
      str  r1, [r0]
```

← Inhalt von R1 an 0x48000814
schreiben, um LEDs einzuschalten



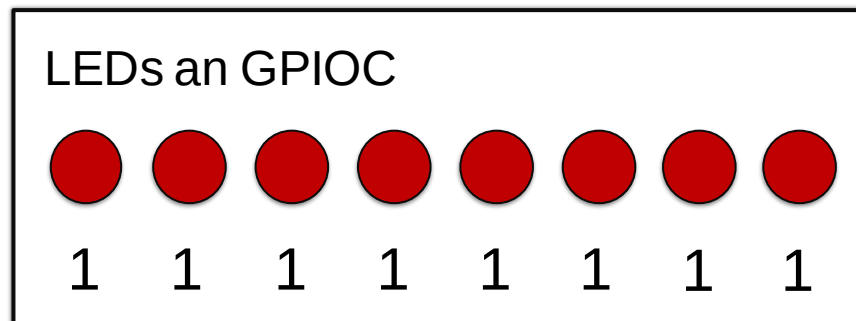
Digitale Ausgabe (Digital Out)

Beispiel: LEDs an GPIOC, Pin 0 bis Pin 7 einschalten

```
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, [r0]
      mov  r2, #0xFF
      orr  r1, r2
      str  r1, [r0]
```

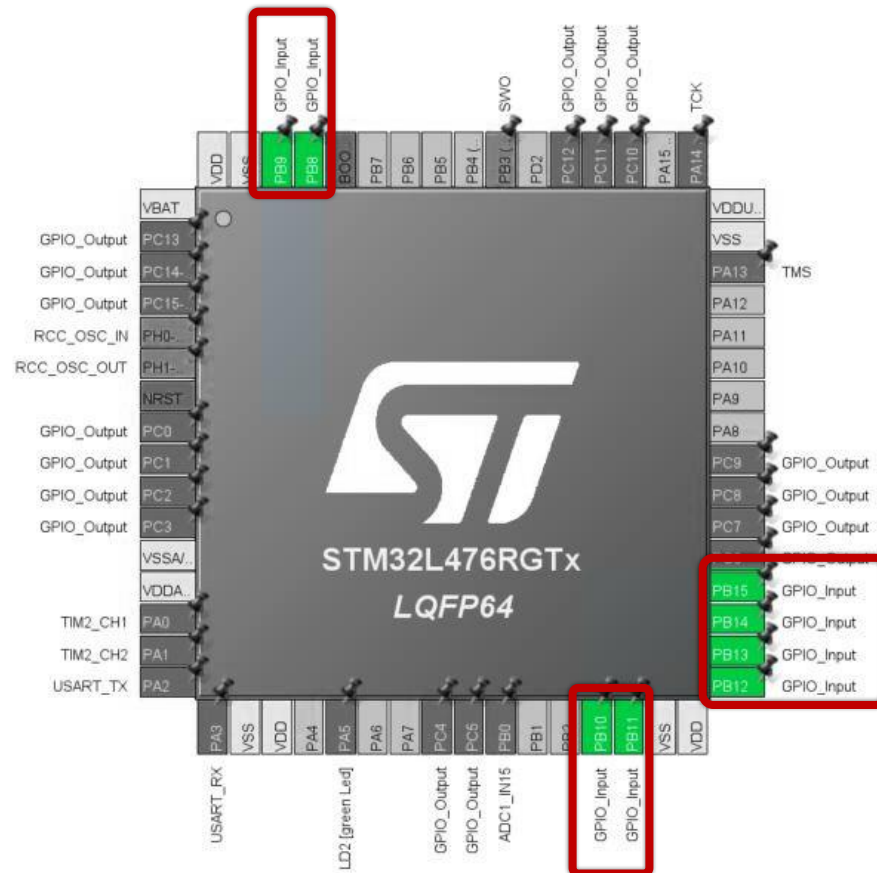
← Inhalt von R1 an 0x48000814
schreiben, um LEDs einzuschalten



Digitale Eingabe (Digital In)

Digitale Eingabe (Digital In)

Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf den LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben



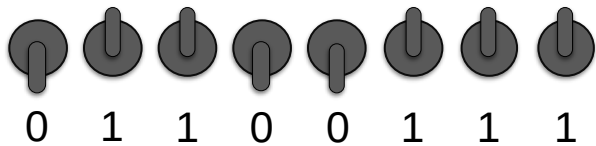
Digitale Eingabe (Digital In)

Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

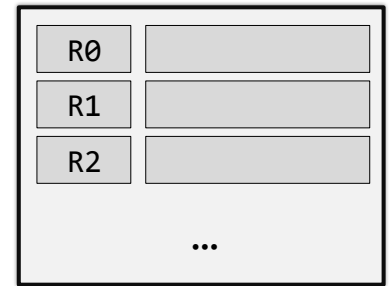
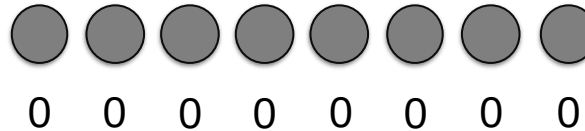
```
.equ GPIOB_IDR, 0x48000410 ← Adresse: GPIOB, Input-Register
.equ GPIOC_ODR, 0x48000814

prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, =#GPIOB_IDR
      ldr  r2, [r1]
      lsr  r3, r2, #8
      str  r3, [r0]
```

Schalter an GPIOB



LEDs an GPIOC



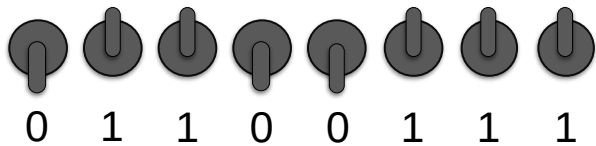
Digitale Eingabe (Digital In)

Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

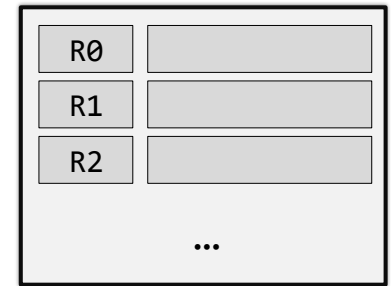
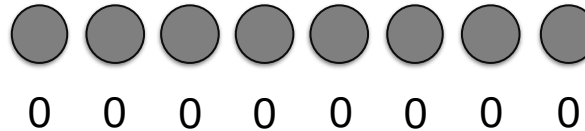
```
.equ GPIOB_IDR, 0x48000410
.equ GPIOC_ODR, 0x48000814 ← Adresse: GPIOC, Output-Register

prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, =#GPIOB_IDR
      ldr  r2, [r1]
      lsr  r3, r2, #8
      str  r3, [r0]
```

Schalter an GPIOB



LEDs an GPIOC



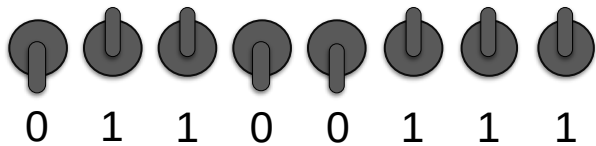
Digitale Eingabe (Digital In)

Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

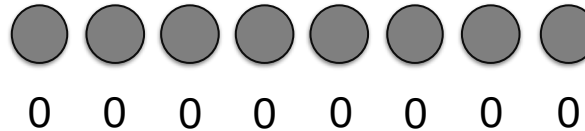
```
.equ GPIOB_IDR, 0x48000410  
.equ GPIOC_ODR, 0x48000814
```

```
prog: ldr  r0, =#GPIOC_ODR ← 0x48000814 in R0  
      ldr  r1, =#GPIOB_IDR  
      ldr  r2, [r1]  
      lsr  r3, r2, #8  
      str  r3, [r0]
```

Schalter an GPIOB



LEDs an GPIOC



R0	0x48000814
R1	
R2	
...	

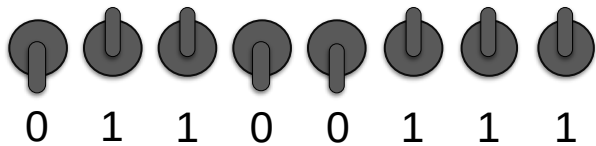
Digitale Eingabe (Digital In)

Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

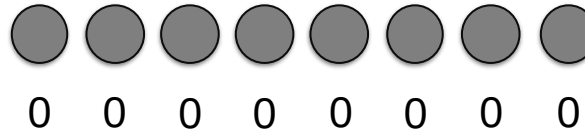
```
.equ GPIOB_IDR, 0x48000410
.equ GPIOC_ODR, 0x48000814

prog: ldr  r0, =#GPIOC_ODR
      ldr  r1, =#GPIOB_IDR ← 0x48000410 in R1
      ldr  r2, [r1]
      lsr  r3, r2, #8
      str  r3, [r0]
```

Schalter an GPIOB



LEDs an GPIOC



R0	0x48000814
R1	0x48000410
R2	
...	

Digitale Eingabe (Digital In)

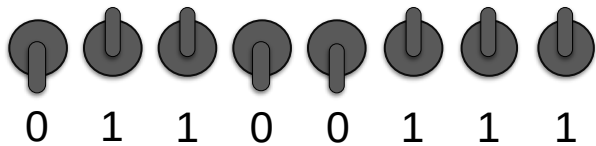
Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

```
.equ GPIOB_IDR, 0x48000410  
.equ GPIOC_ODR, 0x48000814
```

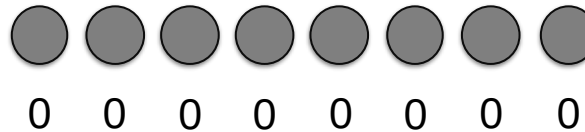
```
prog: ldr  r0, =#GPIOC_ODR  
      ldr  r1, =#GPIOB_IDR  
      ldr  r2, [r1]  
      lsr  r3, r2, #8  
      str  r3, [r0]
```

Schalterstellung in R2

Schalter an GPIOB



LEDs an GPIOC



R0	0x48000814
R1	0x48000410
R2	0x00006700
...	

Digitale Eingabe (Digital In)

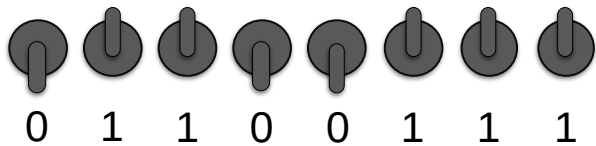
Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

```
.equ GPIOB_IDR, 0x48000410  
.equ GPIOC_ODR, 0x48000814
```

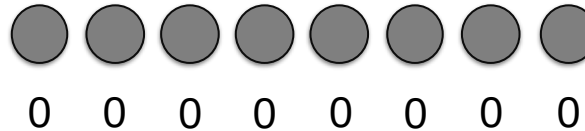
```
prog: ldr  r0, =#GPIOC_ODR  
      ldr  r1, =#GPIOB_IDR  
      ldr  r2, [r1]  
      lsr  r3, r2, #8  
      str  r3, [r0]
```

Logical Shift Left, damit Wert an Pin 0 bis Pin 7 geschrieben werden kann

Schalter an GPIOB



LEDs an GPIOC



R0	0x48000814
R1	0x48000410
R2	0x00000067
...	

Digitale Eingabe (Digital In)

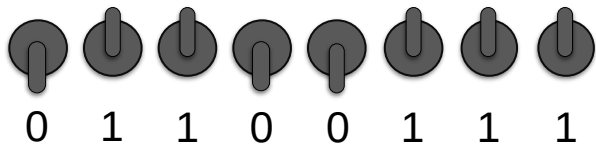
Beispiel: Schalter an GPIOB, Pin 8 bis 15 einlesen und auf LEDs an GPIOC, Pin 0 bis Pin 7 ausgeben

```
.equ GPIOB_IDR, 0x48000410  
.equ GPIOC_ODR, 0x48000814
```

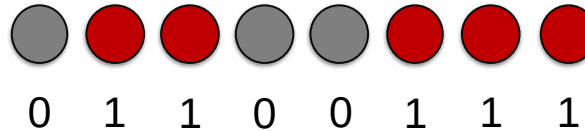
```
prog: ldr  r0, =#GPIOC_ODR  
      ldr  r1, =#GPIOB_IDR  
      ldr  r2, [r1]  
      lsr  r3, r2, #8  
      str  r3, [r0]
```

← Wert an LEDs schreiben

Schalter an GPIOB



LEDs an GPIOC

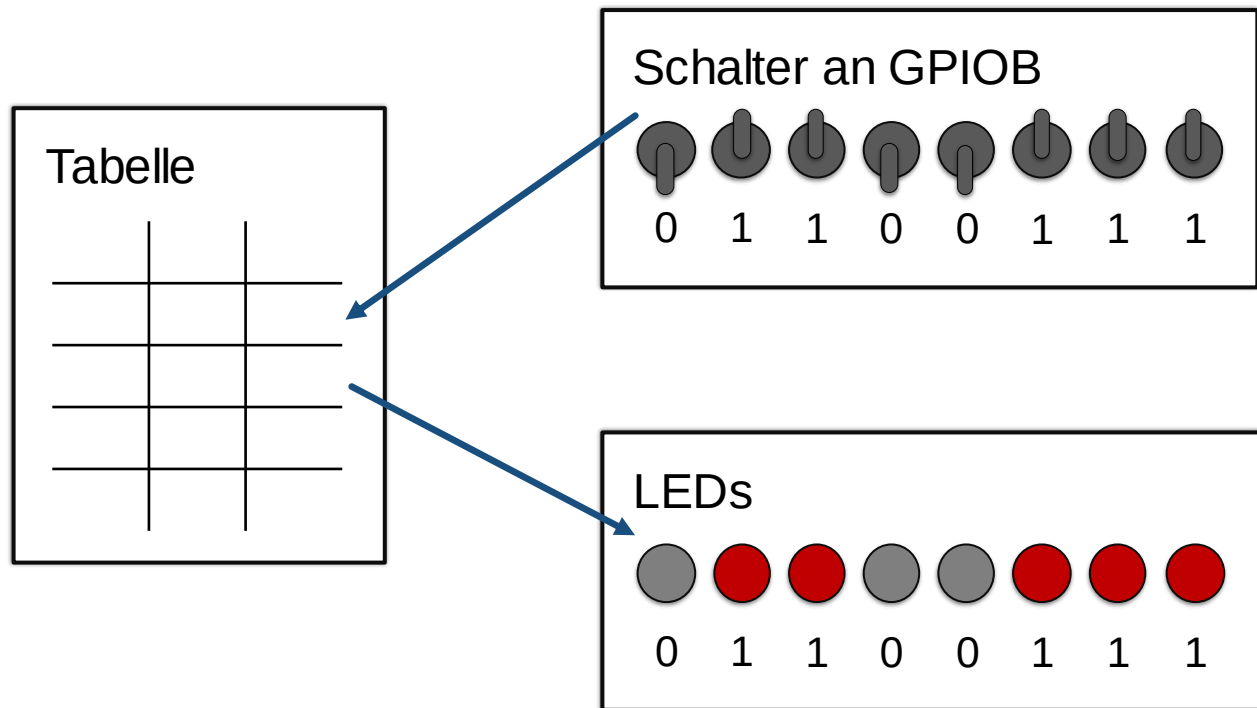


R0	0x48000814
R1	0x48000410
R2	0x00000067
...	

Beispiel: Tabellenverarbeitung

Beispiel: Tabellenverarbeitung

Das Byte aus der Tabelle holen, dessen Position durch die Schalterstellung bestimmt ist.



Wiederholung: Adressierungsarten

R0	0x20000010
R1	0x0000000C
R2	0x00000042
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Register Offset

ldr r2, [r0, r1]

effektive Adresse:

$$\begin{array}{r} 0x20000010 \\ + 0x0000000C \\ \hline 0x2000001C \end{array}$$

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	0x00000042
...	...	...	...	...

Beispiel: Tabellenverarbeitung

Zugriff auf ein Tabellenelement, dessen Tabellenplatz erst zur Laufzeit bestimmt wird.

Tabelle	0x0A	0x1B	0x2C	0x3D	0x4E	...
Tabellen- element Nr.	0	1	2	3	4	...

Beispiel: Tabellenverarbeitung

Arbeiten mit Tabellen

prog: adr r0, tab ← Pointer auf Tabellenanfang

...

Nr. des Tabellenplatzes berechnen (in R1)

...

ldr r2, [r0, r1]

...

tab: .byte 0x0A, 0x1B, 0x2C, 0x3D, 0x4E



Beispiel: Tabellenverarbeitung

Arbeiten mit Tabellen

```
prog: adr  r0, tab
```

```
...
```

Nr. des Tabellenplatzes berechnen (in R1)

```
...
```

```
ldr  r2, [r0, r1]  Tabellenanfang + 0x3
```

```
...
```

```
tab: .byte  0x0A, 0x1B, 0x2C, 0x3D, 0x4E
```

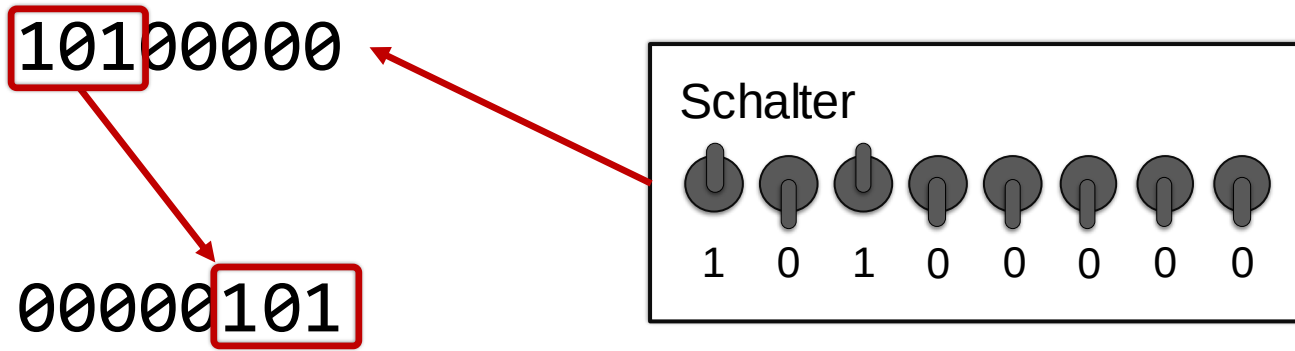
z.B. drittes Element in der
Tabelle: 0x3 in Register R1



Logische Befehle

Schiebebefehle

mit Bitgruppen arbeiten, die kleiner als 8 Bit sind



Schiebebefehle

<code>asr{s}{cond}</code>	<code>Rd, Rm, Rs</code>	Arithmetic Shift Right (n from 1 to 32)
<code>asr{s}{cond}</code>	<code>Rd, Rm, #n</code>	
<code>lsl{s}{cond}</code>	<code>Rd, Rm, Rs</code>	Logical Shift Left (n from 0 to 32)
<code>lsl{s}{cond}</code>	<code>Rd, Rm, #n</code>	
<code>lsr{s}{cond}</code>	<code>Rd, Rm, Rs</code>	Logical Shift Right (n from 1 to 32)
<code>lsr{s}{cond}</code>	<code>Rd, Rm, #n</code>	
<code>ror{s}{cond}</code>	<code>Rd, Rm, Rs</code>	Rotate Right (n from 1 to 31)
<code>ror{s}{cond}</code>	<code>Rd, Rm, #n</code>	

Schiebebefehle

ASR, LSL, LSR, ROR, and RRX

Arithmetic Shift Right, Logical Shift Left, Logical Shift Right, Rotate Right, and Rotate Right with Extend.

Syntax

```
op{S}{cond} Rd, Rm, Rs
```

```
op{S}{cond} Rd, Rm, #n
```

```
RRX{S}{cond} Rd, Rm
```

Where:

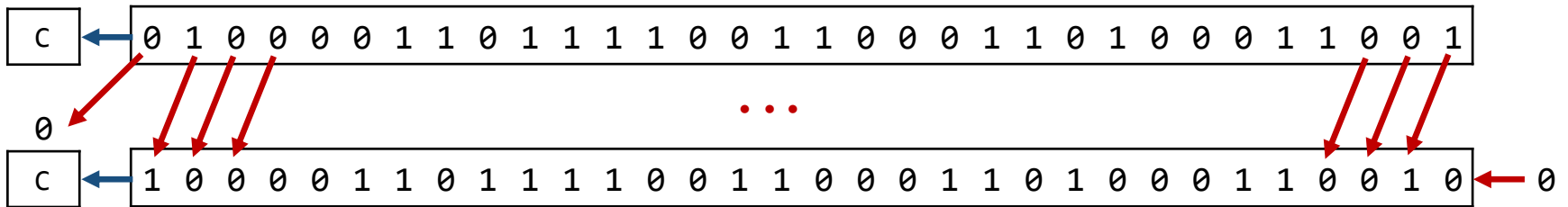
- 'op' is one of the following:
ASR: Arithmetic Shift Right
LSL: Logical Shift Left
LSR: Logical Shift Right
ROR: Rotate Right
- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation, see [Conditional execution on page 65](#).
- 'Rd' is the destination register.
- 'Rm' is the register holding the value to be shifted.
- 'Rs' is the register holding the shift length to apply to the value Rm. Only the least significant byte is used and can be in the range 0 to 255.
- 'n' is the shift length. The range of shift lengths depends on the instruction as follows:
ASR: Shift length from 1 to 32
LSL: Shift length from 0 to 31
LSR: Shift length from 1 to 32
ROR: Shift length from 1 to 31

...

Schiebebefehle

LSL{S} Rd, Rm, #n

Register R1



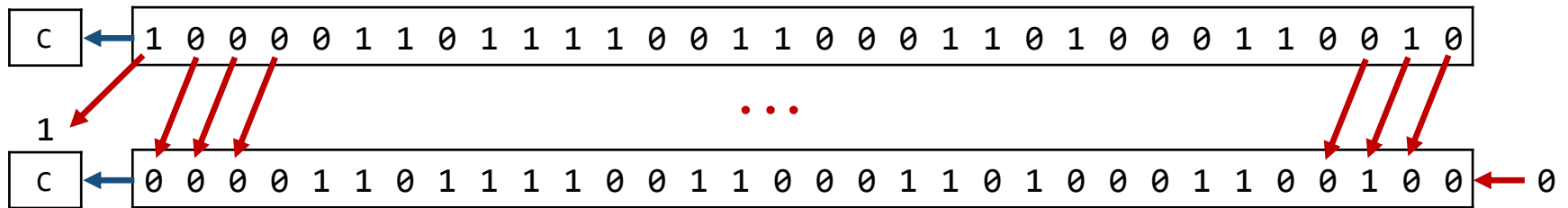
Register R2

lsl r2, r1, #1

Schiebebefehle

LSL{S} Rd, Rm, #n

Register R1



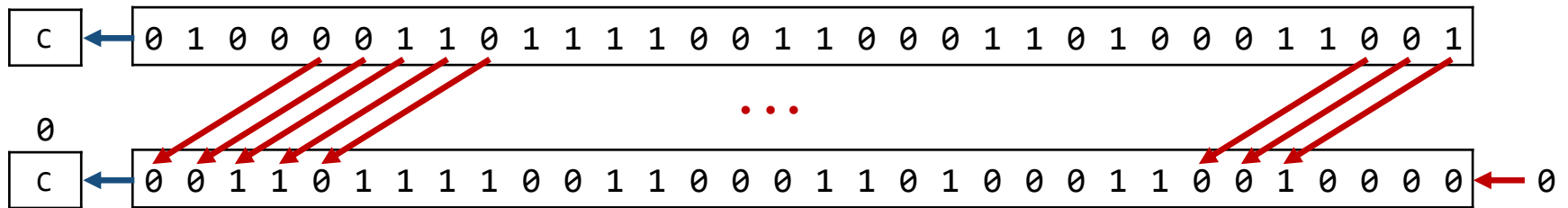
Register R2

lsls r2, r1, #1

Schiebebefehle

LSL{S} Rd, Rm, #n

Register R1



Register R2

lsl r2, r1, #4

Logische Befehle

Problem: Verarbeiten von Objekten, die kleiner als 16 Bit sind

Beispiel: 16 Bit einlesen in Register R0

1010011010110100

Bit 3 bis Bit 7 als Zahl weiter verwenden

10100110**10110**100

Logische Befehle

AND, ORR, EOR, BIC, and ORN

Logical AND, OR, Exclusive OR, Bit Clear, and OR NOT.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

Where:

- 'op' is one of:
 - AND: Logical AND.
 - ORR: Logical OR or bit set.
 - EOR: Logical exclusive OR.
 - BIC: Logical AND NOT or bit clear.
 - ORN: Logical OR NOT.
- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation, see [Conditional execution on page 65](#).
- 'cond' is an optional condition code, see [Conditional execution on page 65](#).
- 'Rd' is the destination register.
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand, see [Flexible second operand on page 60](#) for details of the options.

...

Logische Befehle

AND, ORR, EOR, BIC, and ORN

Logical AND, OR, Exclusive OR, Bit Clear, and OR NOT.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

Where:

- 'op' is one of:
 - AND: Logical AND.
 - ORR: Logical OR or bit set.
 - EOR: Logical exclusive OR.
 - BIC: Logical AND NOT or bit clear.
 - ORN: Logical OR NOT.
- 'S' is an optional suffix. If S is specified, result of the operation, see [Conditional](#)
- 'cond' is an optional condition code, see [page 65](#).
- 'Rd' is the destination register.
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand, see [Flexible second operand on page 60](#) for details of the options.

0011

0101

0001

updated on the

[page 65](#).

...

Logische Befehle

Problem: Verarbeiten von Objekten, die kleiner als 16 Bit sind

Beispiel: 16 Bit einlesen in Register R0 1010011010110100

Bit 3 bis Bit 7 als Zahl weiter verwenden 10100110**10110**100

```
mov  r0, #0xA6B4
mov  r1, #0xF8
and  r2, r1, r0
```

R0 vor AND: 1010011010110100

Maske: 0000000011111000

R0 nach AND: 0000000010110000

Logische Befehle

AND, ORR, EOR, BIC, and ORN

Logical AND, OR, Exclusive OR, Bit Clear, and OR NOT.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

Where:

- 'op' is one of:
 - AND: Logical AND.
 - ORR: Logical OR or bit set.
 - EOR: Logical exclusive OR.
 - BIC: Logical AND NOT or bit clear.
 - ORN: Logical OR NOT.
- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation, see [Conditional execution on page 65](#).
- 'cond' is an optional condition code, see [Conditional execution on page 65](#).
- 'Rd' is the destination register.
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand, see [Flexible second operand on page 60](#) for details of the options.

...

Logische Befehle

AND, **ORR**, EOR, BIC, and ORN

Logical AND, OR, Exclusive OR, Bit Clear, and OR NOT.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

Where:

- 'op' is one of:
 - AND: Logical AND.
 - ORR: Logical OR or bit set.
 - EOR: Logical exclusive OR.
 - BIC: Logical AND NOT or bit clear.
 - ORN: Logical OR NOT.
- 'S' is an optional suffix. If S is specified, result of the operation, see [Conditional](#)
- 'cond' is an optional condition code, see [page 65](#).
- 'Rd' is the destination register.
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand, see [Flexible second operand on page 60](#) for details of the options.

0011

0101

0111

updated on the

[page 65](#).

...

Logische Befehle

AND Bitgruppen oder Einzelbits auf 0 setzen

ORR Bitgruppen oder Einzelbits auf 1 setzen

Beispiel: Verzweigen im Programm, wenn an Bit 0 eines Digitalports eine '1' gelesen wird

```
ldr    r0, ... ← zu prüfender Wert in R0
mov    r1, #1
and    r2, r1, r0
bne    ziel
...
```

```
ziel: ...
```

Logische Befehle

AND Bitgruppen oder Einzelbits auf 0 setzen

ORR Bitgruppen oder Einzelbits auf 1 setzen

Beispiel: Verzweigen im Programm, wenn an Bit 0 eines Digitalports eine '1' gelesen wird

```
ldr    r0, ...  
mov    r1, #1 ← 1 in R1  
and    r2, r1, r0  
bne    ziel  
...
```

```
ziel: ...
```

Logische Befehle

AND Bitgruppen oder Einzelbits auf 0 setzen

ORR Bitgruppen oder Einzelbits auf 1 setzen

Beispiel: Verzweigen im Programm, wenn an Bit 0 eines Digitalports eine '1' gelesen wird

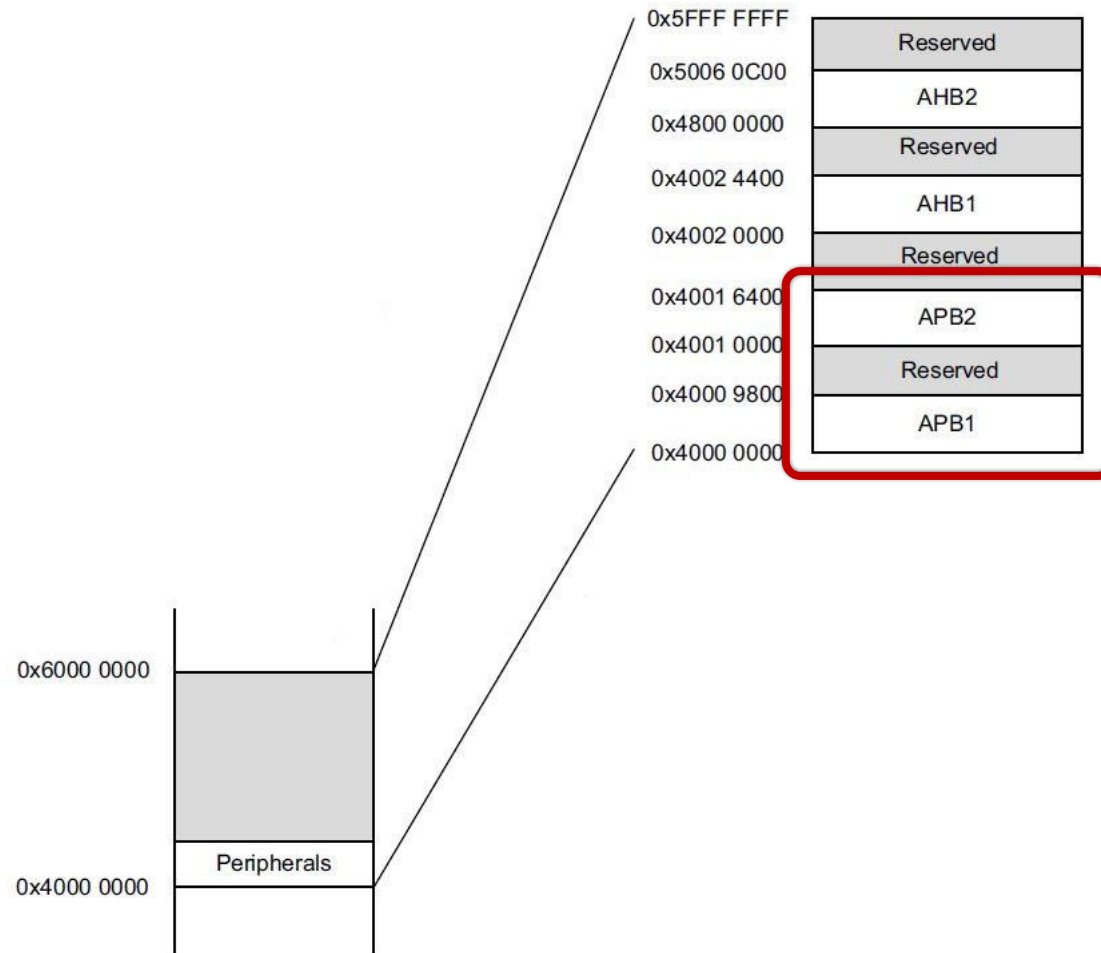
```
ldr    r0, ...  
mov    r1, #1  
and    r2, r1, r0  
bne    ziel  
...  
ziel: ...
```

Diagram illustrating a branch instruction:

- A red arrow labeled **0x01** points from the **bne** instruction to the **ziel** label, indicating a branch taken when the bit is 1.
- A red arrow labeled **0x00** points from the **...** line down, indicating the path when the bit is 0.

Erweiterte On-Chip-Peripherie

On-Chip-Peripherie



On-Chip-Peripherie

0x5FFF FFFF
0x5006 0C00
0x4800 0000
0x4002 4400
0x4002 0000
0x4001 6400
0x4001 0000
0x4000 9800
0x4000 0000

Reserved
AHB2
Reserved
AHB1
Reserved
APB2
Reserved
APB1

Timers

APB2

...		
0x4001 4400 - 0x4001 47FF	1 KB	TIM16
0x4001 4000 - 0x4001 43FF	1 KB	TIM15
0x4001 3C00 - 0x4001 3FFF	1 KB	Reserved
0x4001 3800 - 0x4001 3BFF	1 KB	USART1
0x4001 3400 - 0x4001 37FF	1 KB	TIM8
0x4001 3000 - 0x4001 33FF	1 KB	SPI1
0x4001 2C00 - 0x4001 2FFF	1 KB	TIM1
0x4001 2800 - 0x4001 2BFF	1 KB	SDMMC1
0x4001 2000 - 0x4001 27FF	2 KB	Reserved
...		

On-Chip-Peripherie

Reference Manual: Timers

TIM1/TIM8 introduction

The advanced-control timers (TIM1/TIM8) consist of a 16-bit auto-reload counter driven by a programmable prescaler.

It may be used for a variety of purposes, including measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare, PWM, complementary PWM with dead-time insertion).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

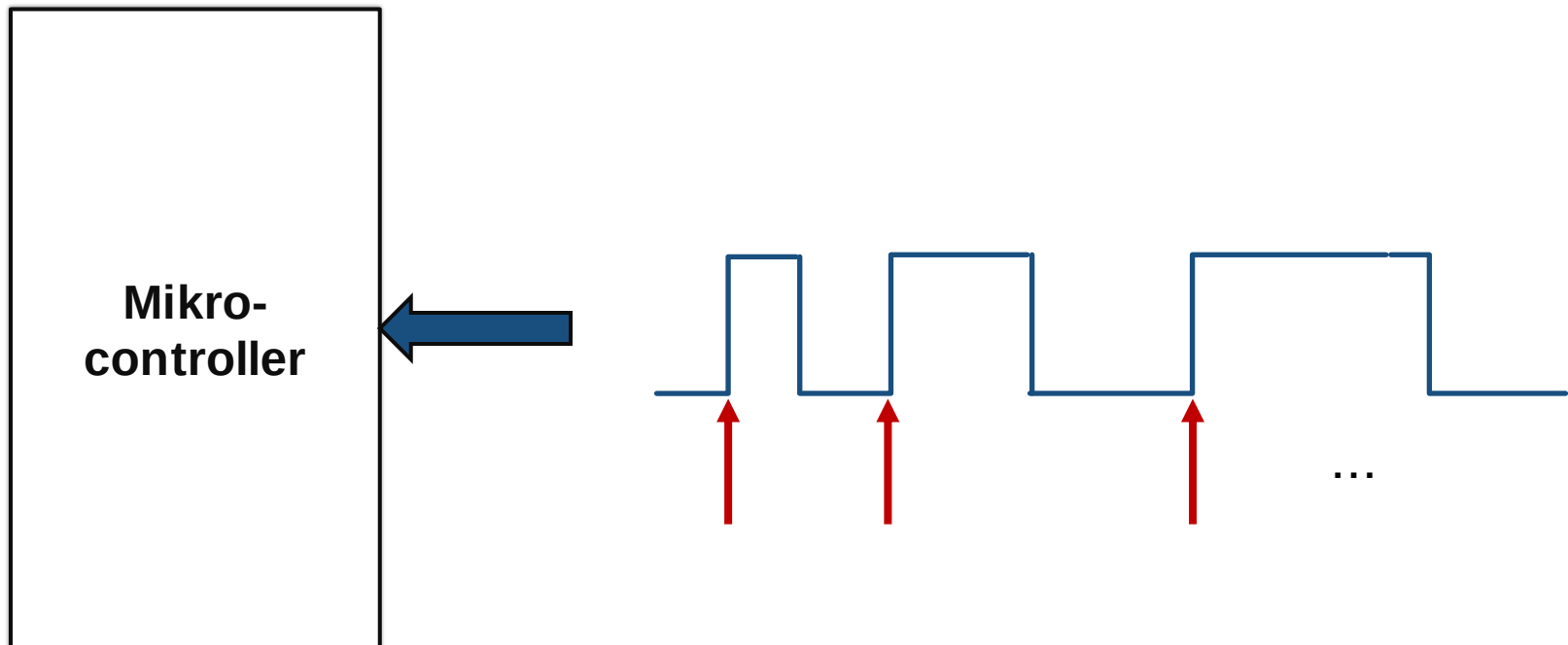
The advanced-control (TIM1/TIM8) and general-purpose (TIMy) timers are completely independent, and do not share any resources. They can be synchronized together as described in [Section 30.3.26: Timer synchronization](#).

...

On-Chip-Peripherie

Beispiel: Input-Capture

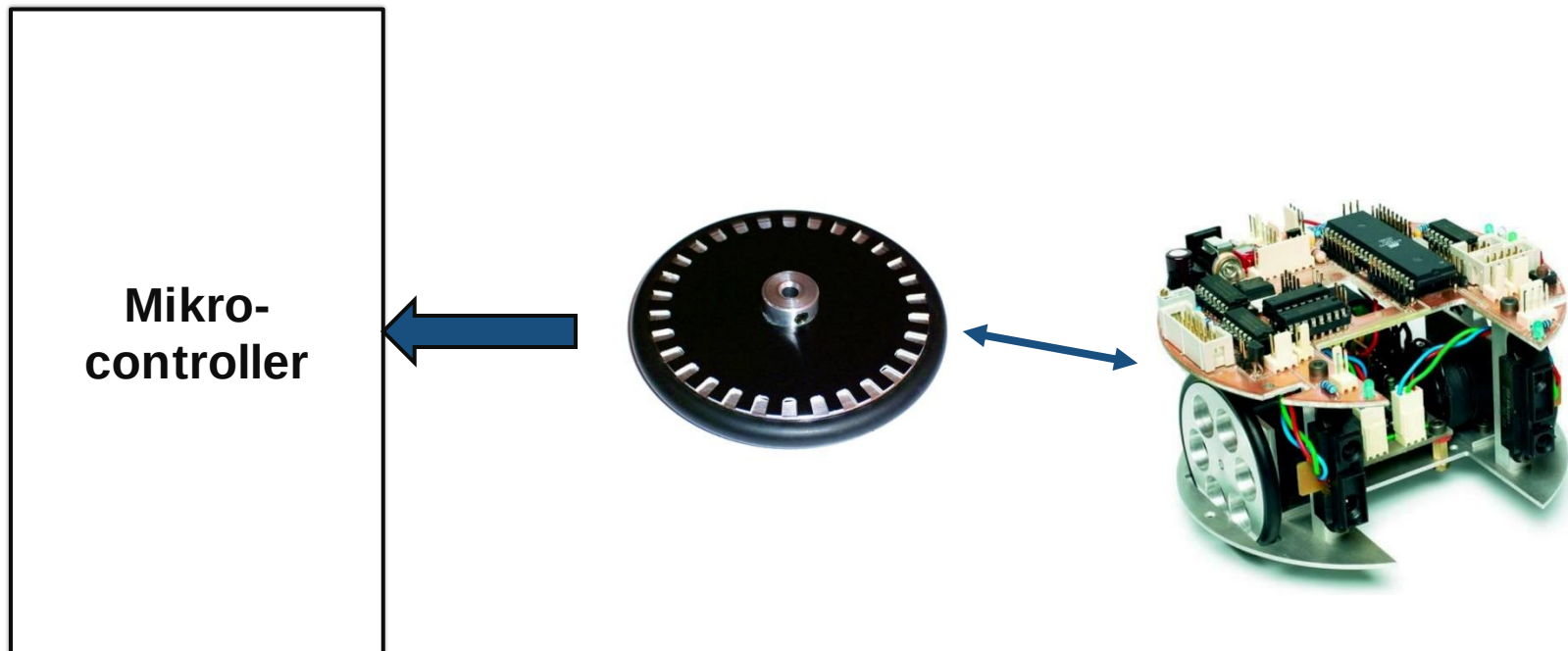
Periodenlängen / Impulslängen bestimmen



On-Chip-Peripherie

Beispiel: Input-Capture

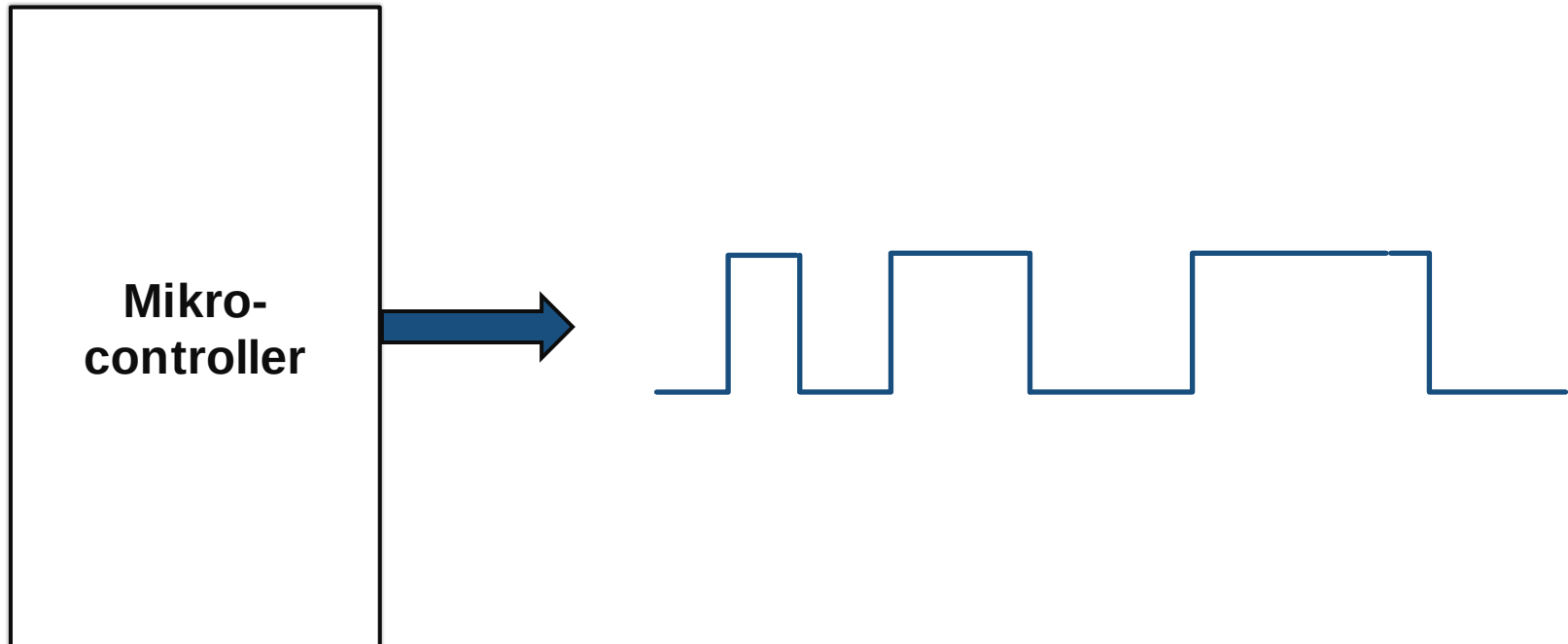
Rad-Encoder bei mobilen Robotern



On-Chip-Peripherie

Beispiel: Output-Compare

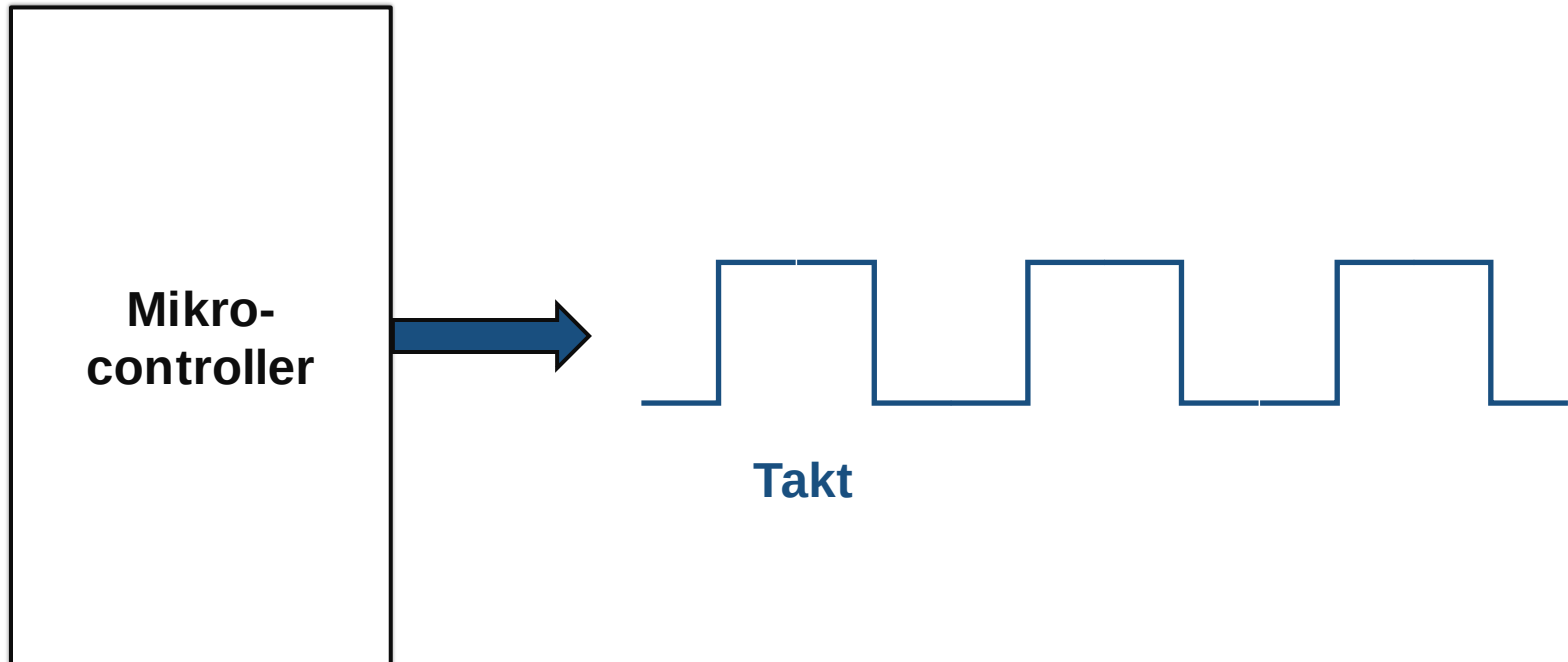
periodische und nicht-periodische Signale erzeugen



On-Chip-Peripherie

Beispiel: PWM

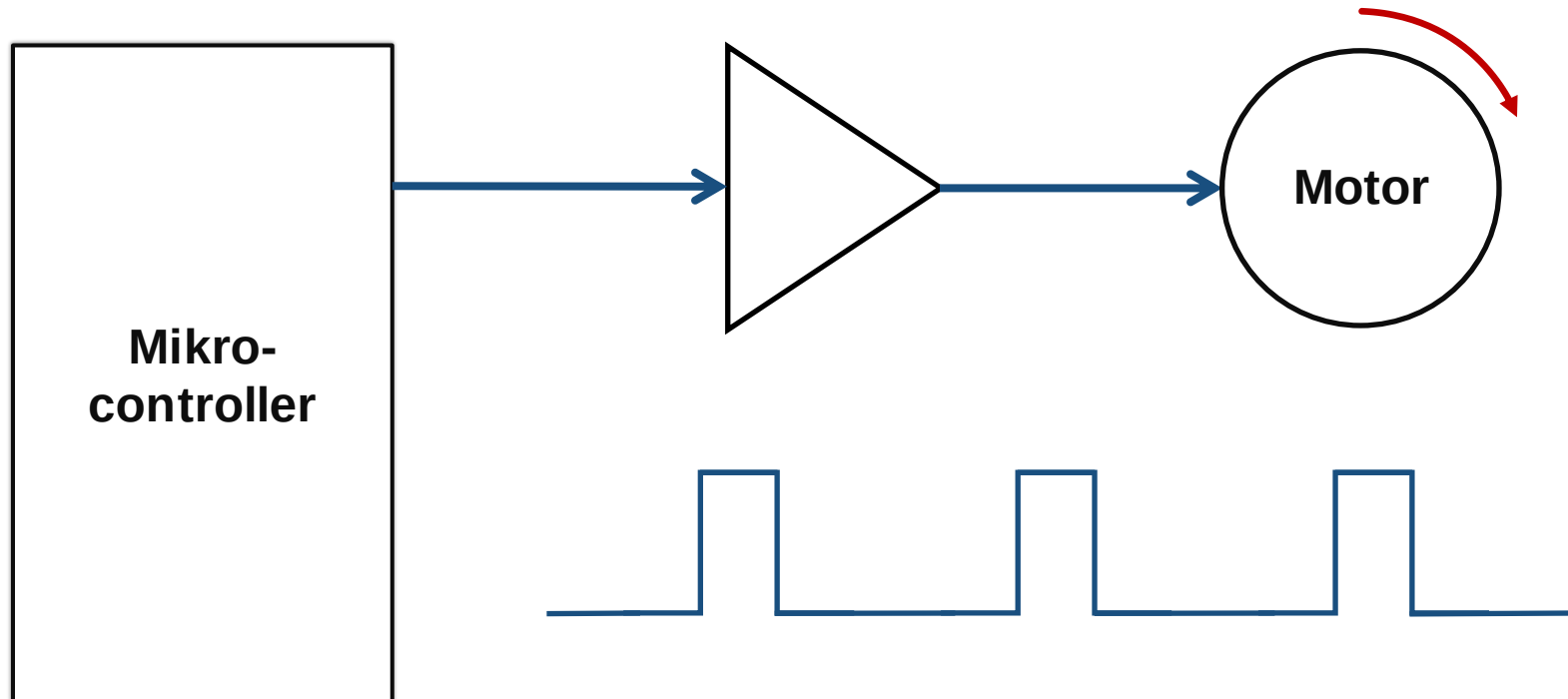
periodische Takt-Signale erzeugen



On-Chip-Peripherie

Beispiel: PWM

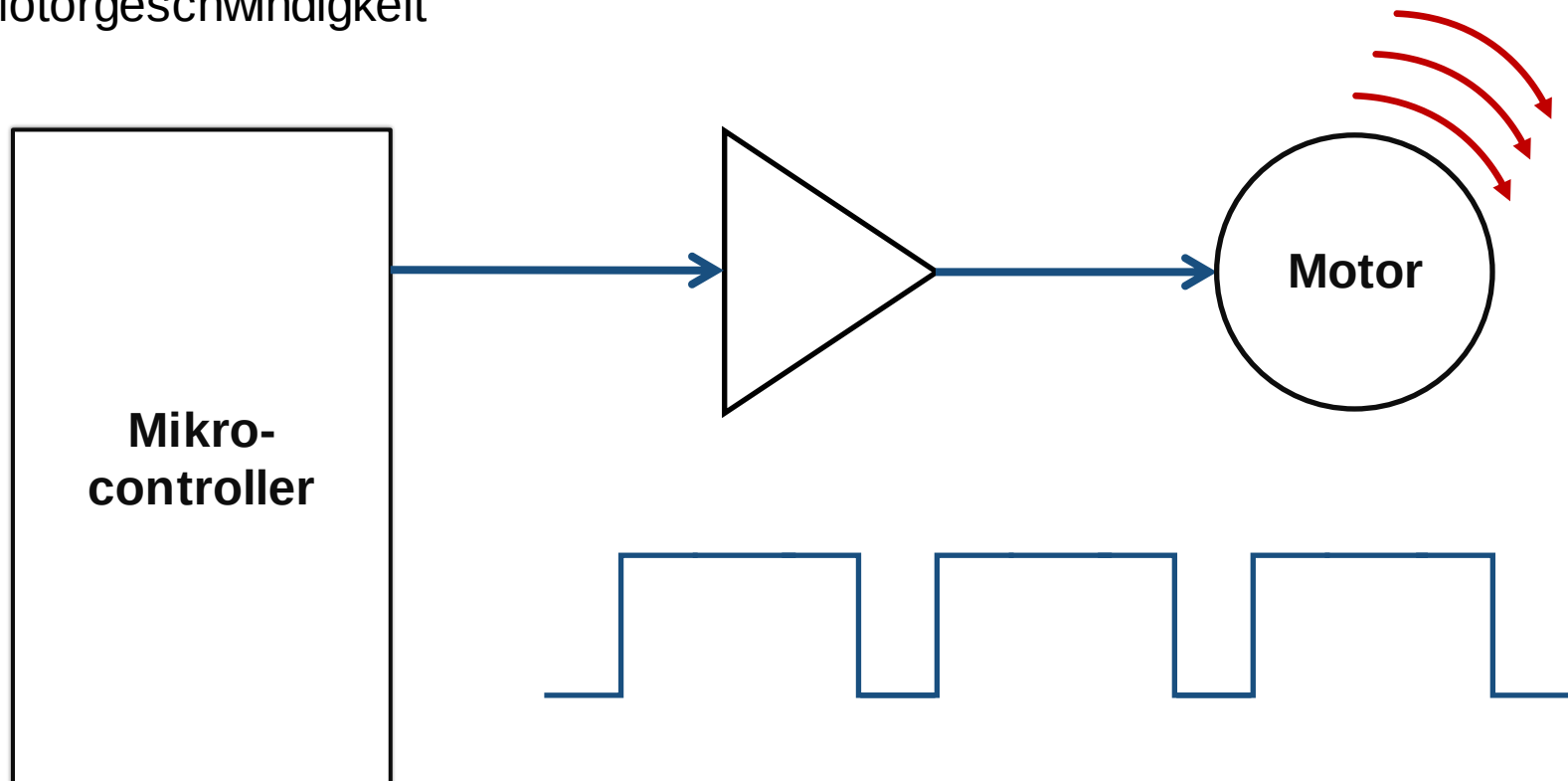
Motorgeschwindigkeit



On-Chip-Peripherie

Beispiel: PWM

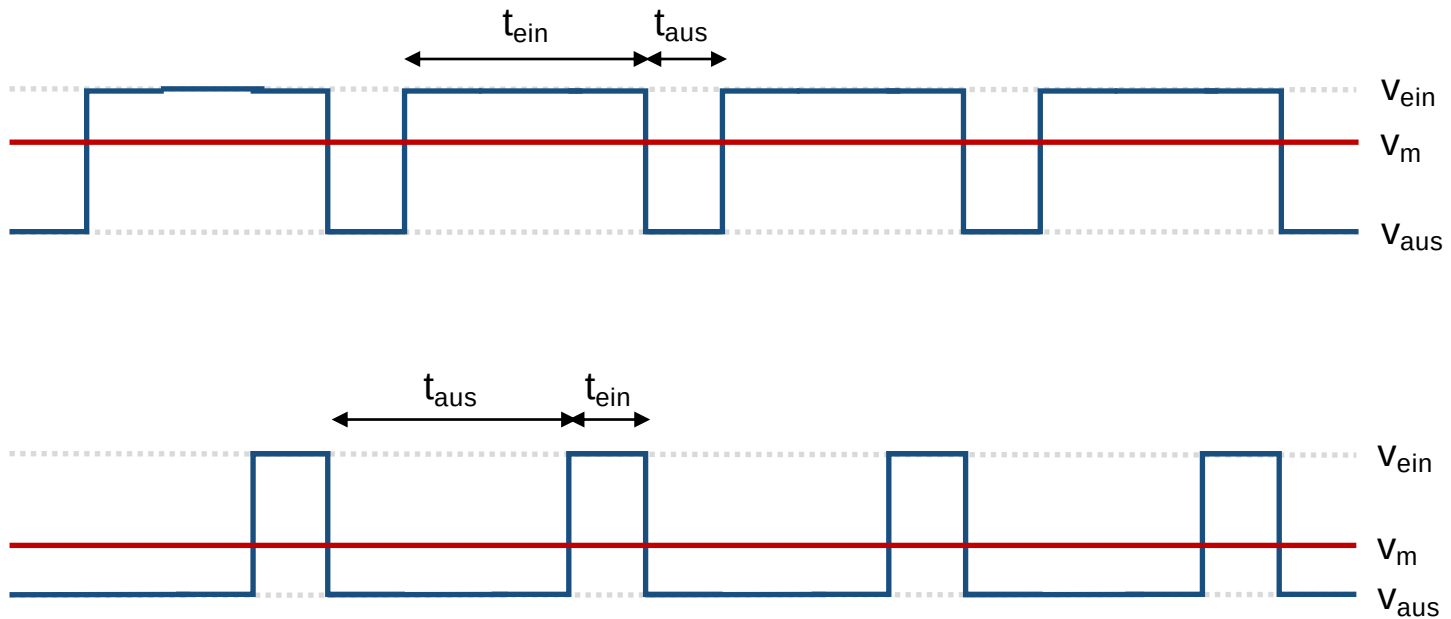
Motorgeschwindigkeit



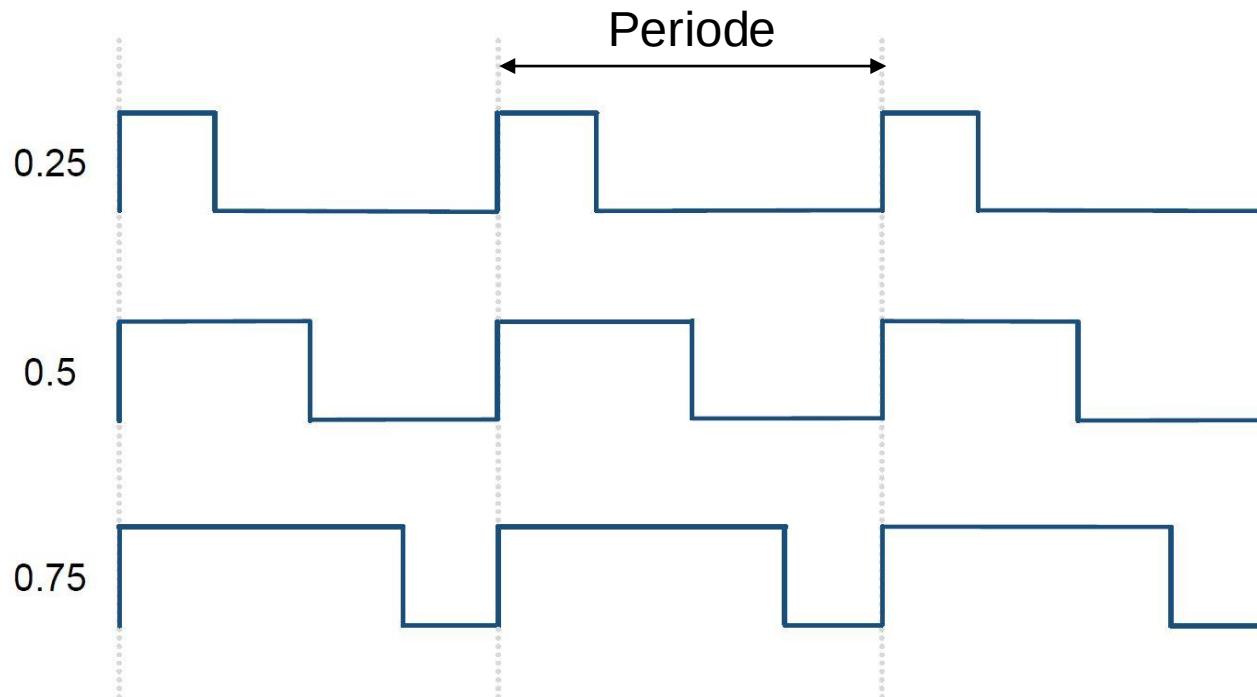
On-Chip-Peripherie

Beispiel: PWM

Motorgeschwindigkeit

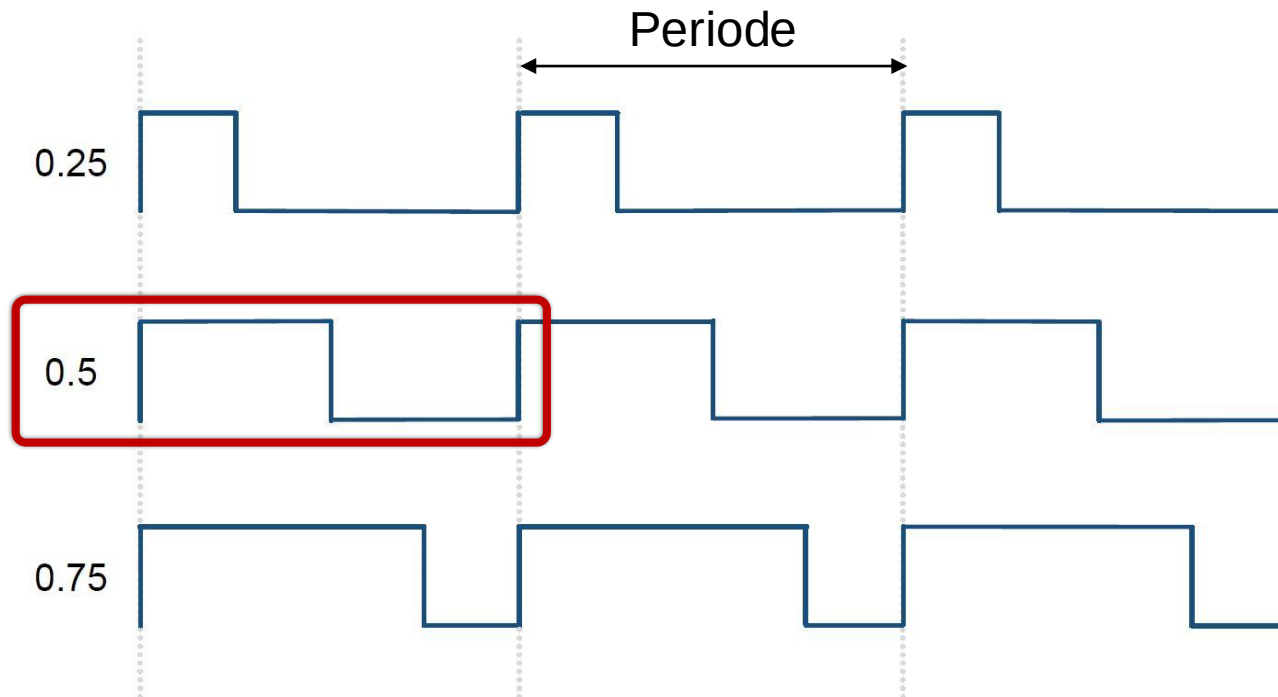


On-Chip-Peripherie



Tastverhältnisse 0.25, 0.5 und 0.75 bei gleicher Frequenz

On-Chip-Peripherie



Tastverhältnisse 0.25, 0.5 und 0.75 bei gleicher Frequenz

On-Chip-Peripherie

Beispiel: Periodendauer 400ns, Low 100ns, High 300ns

